



Programación Funcional

Clases teóricas

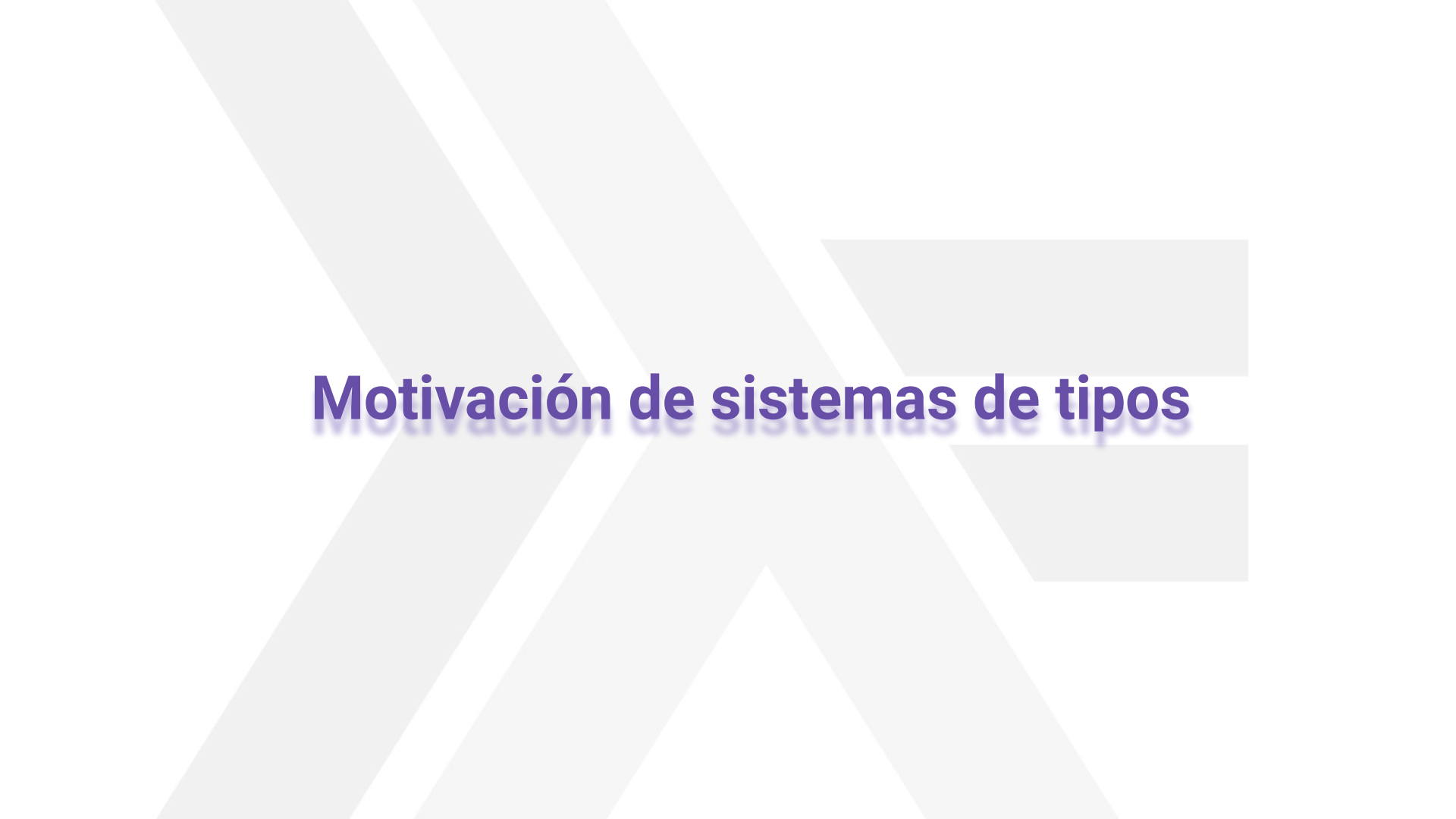
por Pablo E. “Fidel” Martínez López

2. Sistemas de tipos

“Ged, acurrucado entre unos matorrales, mojado y melancólico, se preguntaba de qué servía tener poder si una prudencia excesiva impedía utilizarlo, y lamentaba no haber entrado de aprendiz del hechicero del Valle, donde al menos hubiera podido dormir seco.”

Un mago de Terramar
Úrsula K. Le Guin





Motivación de sistemas de tipos

Motivación

- ❑ ¿Toda combinación de símbolos es una expresión?

Motivación

- ❑ ¿Toda combinación de símbolos es una expresión?
- ❑ Consideremos
 - ❑ $(2+$

Motivación

- ❑ ¿Toda combinación de símbolos es una expresión?
- ❑ Consideremos
 - ❑ $(2+$
 - ❑ En este ejemplo, faltan símbolos

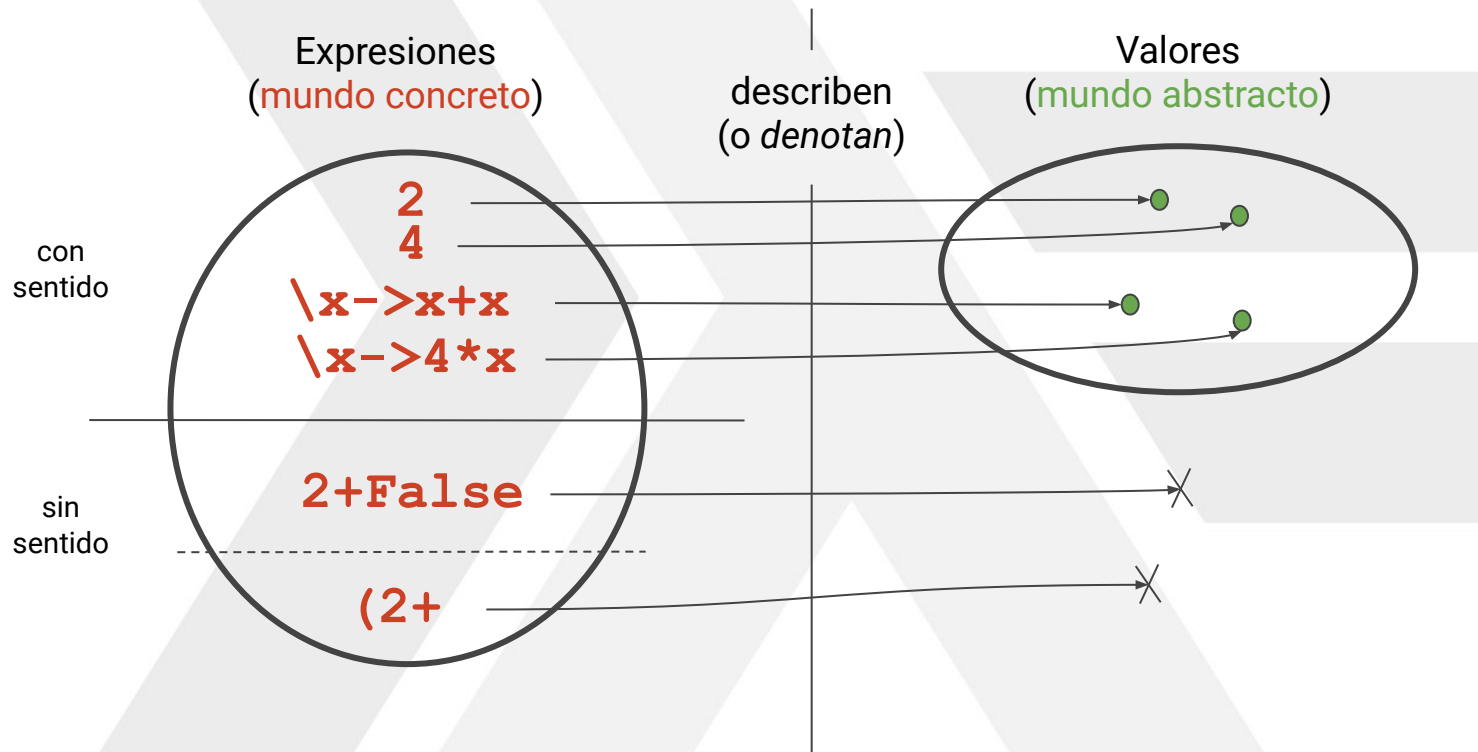
Motivación

- ❑ ¿Toda combinación de símbolos es una expresión?
- ❑ Consideremos
 - ❑ `(2+`
 - ❑ En este ejemplo, faltan símbolos
 - ❑ `2+False`

Motivación

- ❑ ¿Toda combinación de símbolos es una expresión?
- ❑ Consideremos
 - ❑ **(2+**
 - ❑ En este ejemplo, faltan símbolos
 - ❑ **2+False**
 - ❑ Pero este podría tener significado

Motivación



Motivación

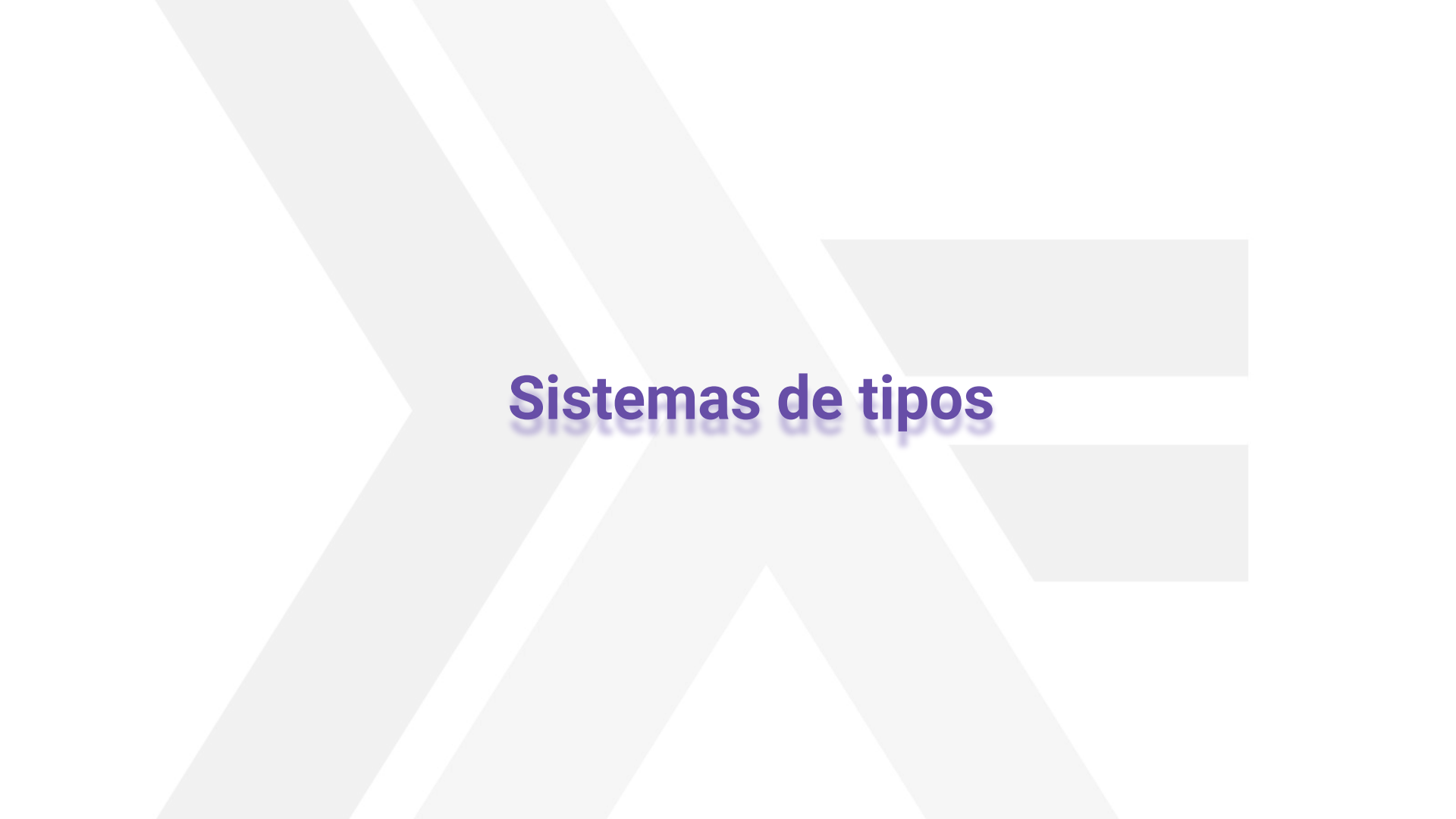
- ❑ ¿Cómo determinar si una expresión es válida?
- ❑ O sea, si tiene sentido

Motivación

- ❑ ¿Cómo determinar si una secuencia de caracteres es una expresión válida?
- ❑ O sea, si es expresión, y si tiene sentido
- ❑ Reglas sintácticas
 - ❑ BNF u otras formas similares

Motivación

- ❑ ¿Cómo determinar si una secuencia de caracteres es una expresión válida?
- ❑ O sea, si es expresión, y si tiene sentido
- ❑ Reglas sintácticas
 - ❑ BNF u otras formas similares
- ❑ Reglas de asignación de tipo
 - ❑ Gran variedad de sistemas y características

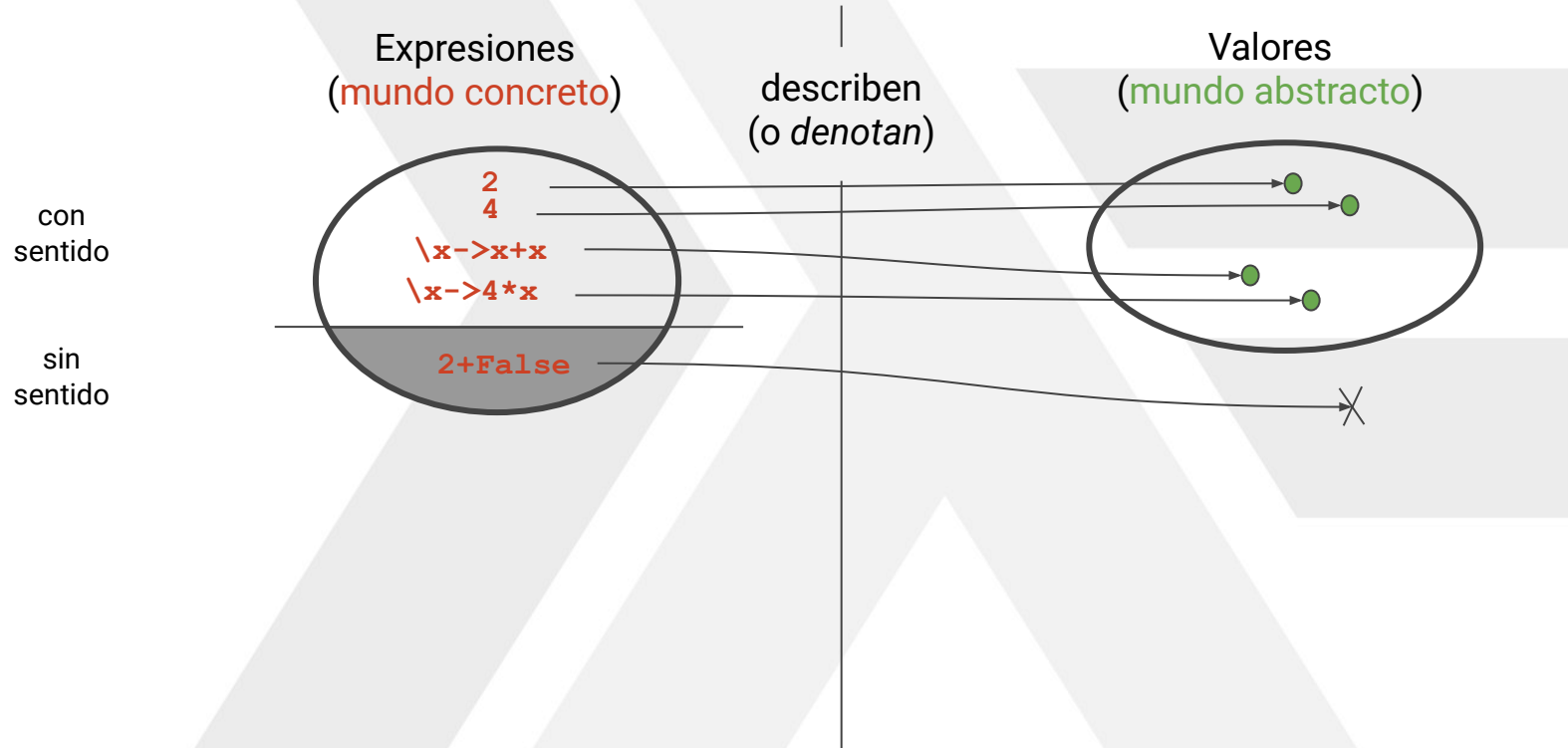


Sistemas de tipos

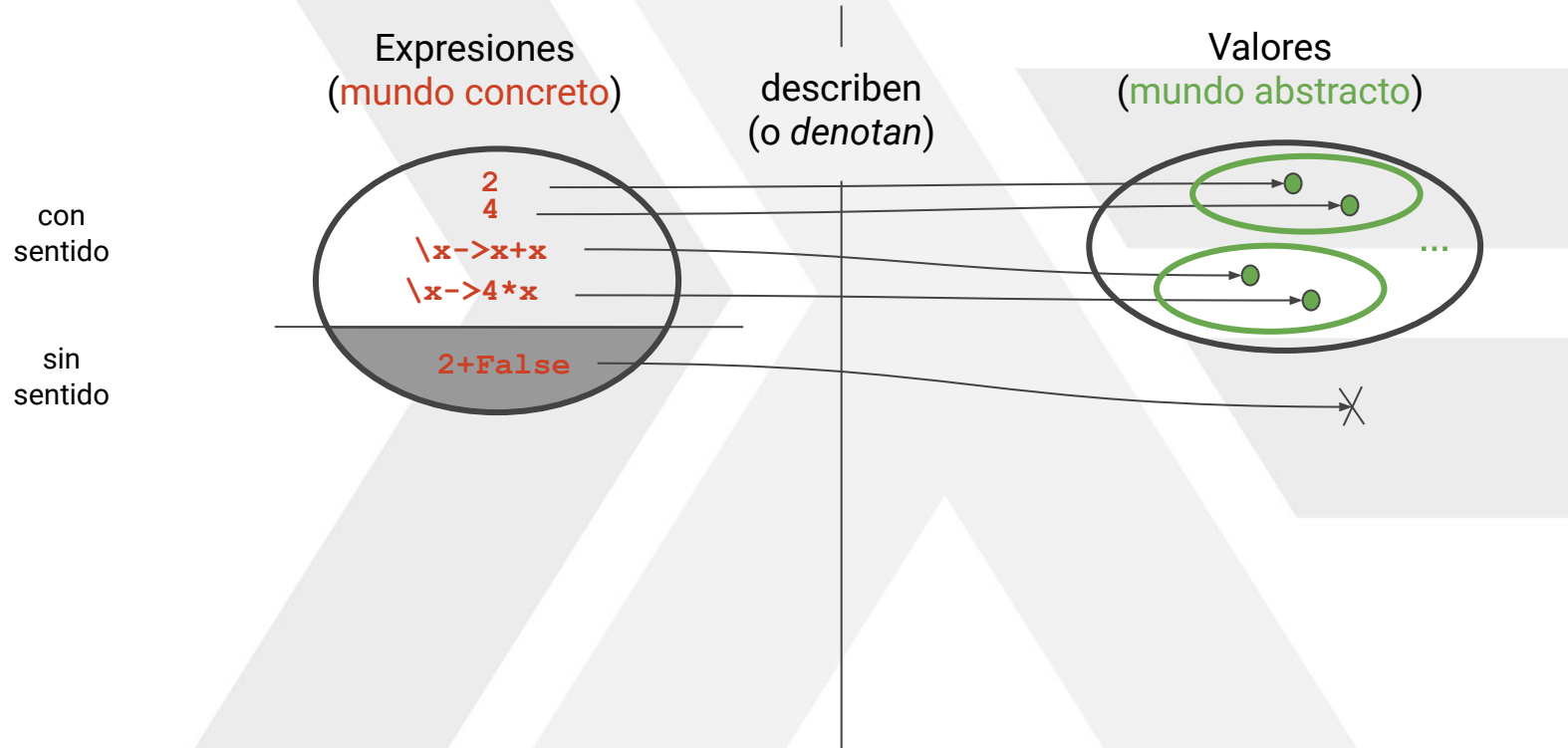
Sistemas de tipos

- ❑ Buscamos un mecanismo para clasificar expresiones en base a su sentido
 - ❑ Expresiones sin sentido no podrán ser clasificadas
- ❑ Es razonable clasificar los valores en conjuntos
 - ❑ Precisamos formas de describir estos conjuntos
 - ❑ Expresiones de tipo

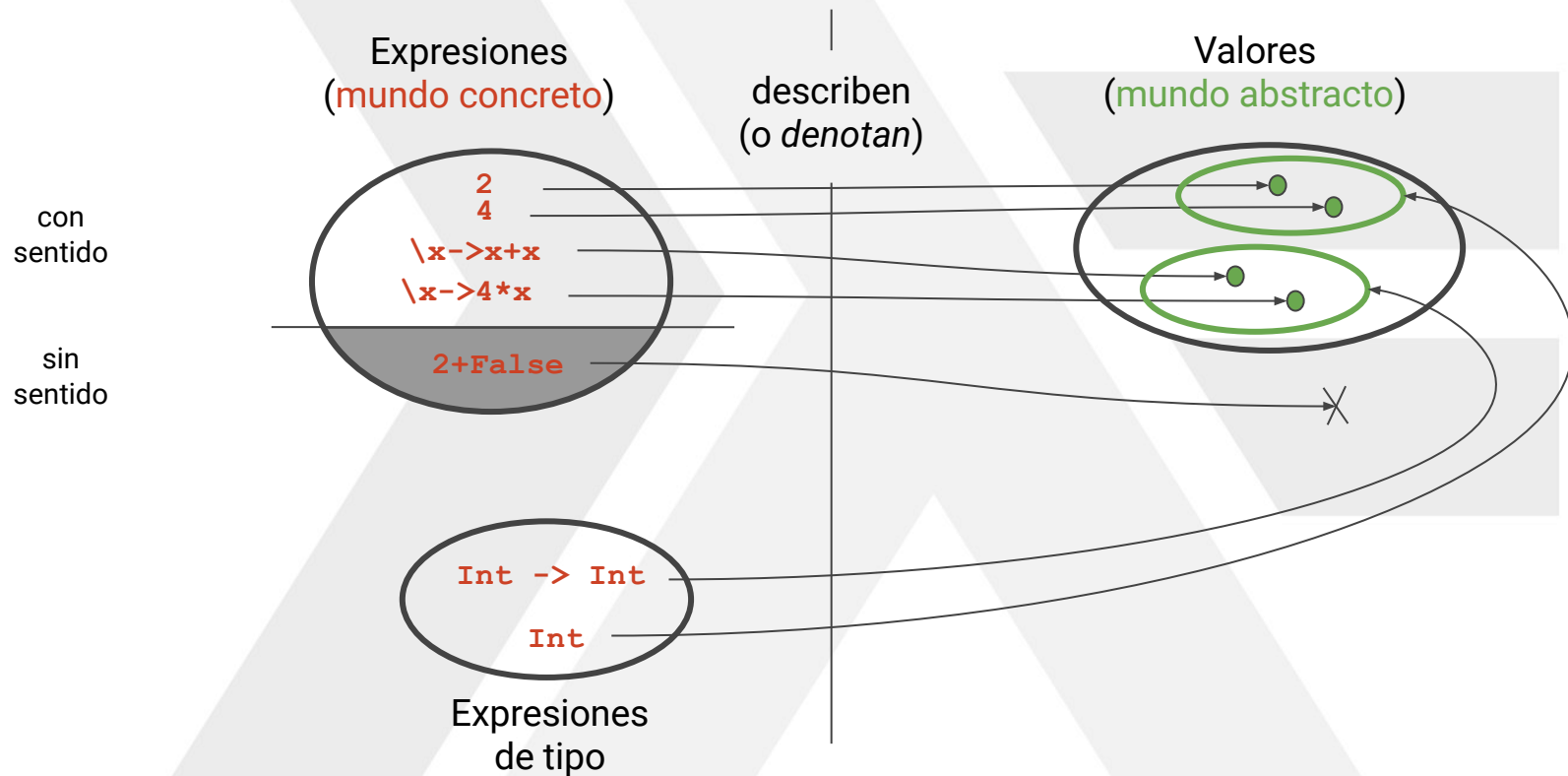
Sistemas de tipos



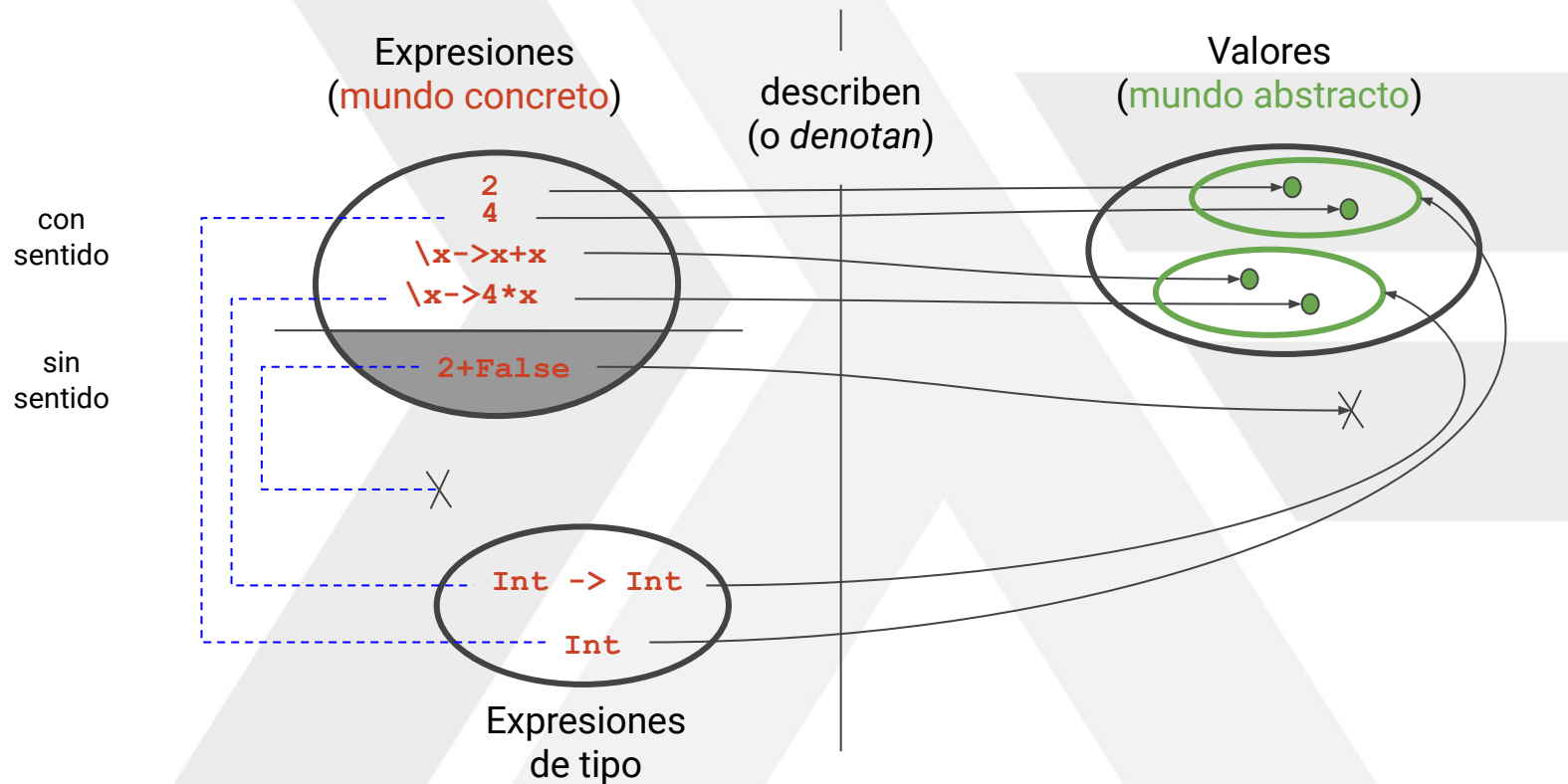
Sistemas de tipos



Sistemas de tipos



Sistemas de tipos



Sistemas de tipos

- ❑ Podemos establecer una relación entre expresiones de valores y expresiones de tipo

Sistemas de tipos

- ❑ Podemos establecer una relación entre expresiones de valores y expresiones de tipo
- ❑ ¿Cómo podemos escribir esta relación?

Sistemas de tipos

- Podemos establecer una relación entre expresiones de valores y expresiones de tipo
- ¿Cómo podemos escribir esta relación?
 - ¡Más símbolos!
 - `2 :: Int`
 - `\x->x+x :: Int -> Int`

Sistemas de tipos

- ❑ La notación es $e :: A$
- ❑ Se lee así: “la expresión e tiene tipo A ”
- ❑ Significa que el valor denotado por e pertenece al conjunto denotado por A
- ❑ Las variables **en esta tipografía** son especiales
 - ❑ se reemplazan por una expresión válida del conjunto indicado
 - ❑ se llaman **metavariables** porque se usan para hablar del lenguaje y NO SON variables del lenguaje

Sistemas de tipos

- ❑ ¿Cómo establecer cuándo se cumple esta relación?
- ❑ Sistemas de reglas

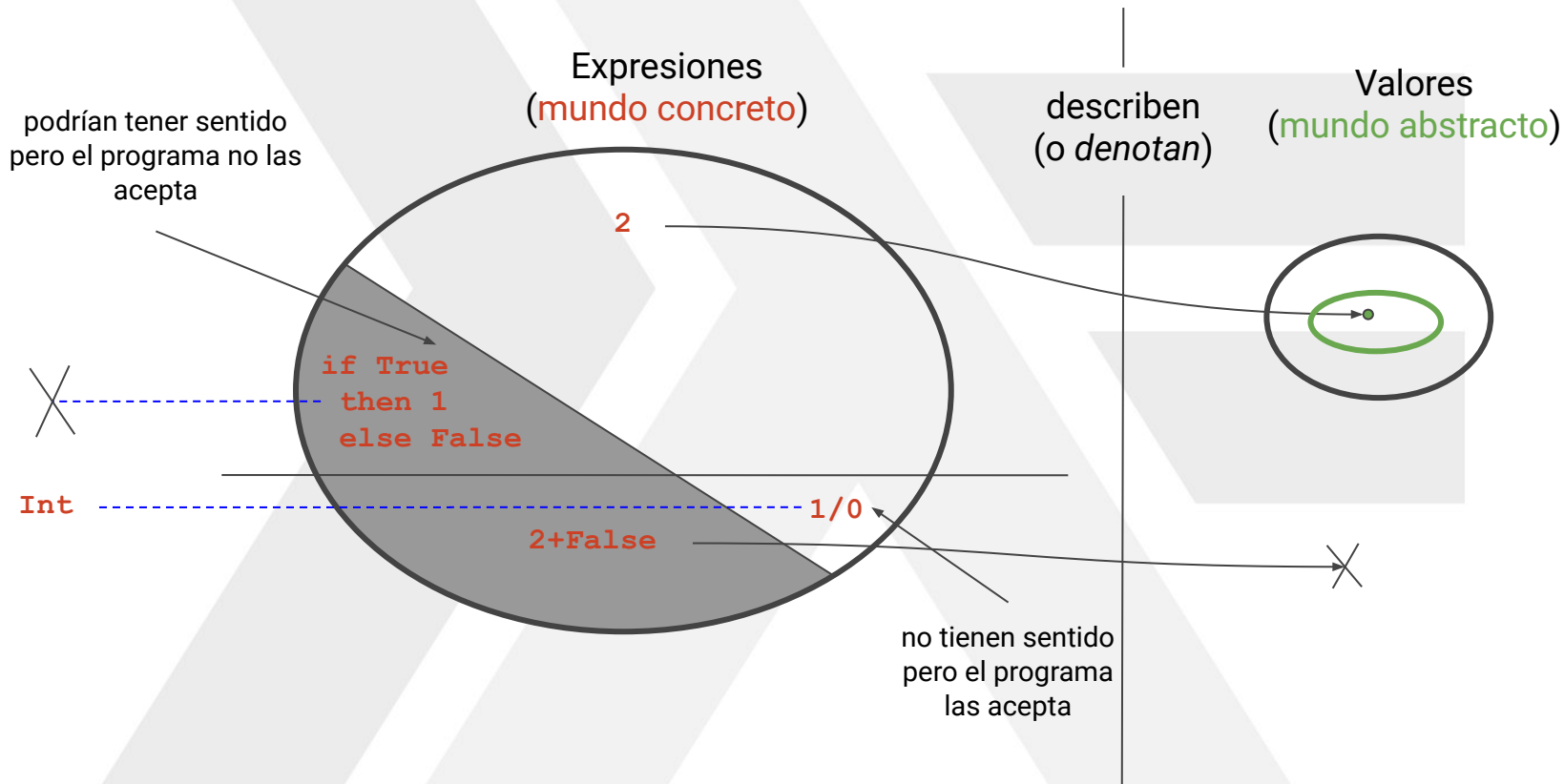
Sistemas de tipos

- ❑ ¿Cómo establecer cuándo se cumple esta relación?
 - ❑ Sistemas de reglas
 - ❑ Si se permite que no haya un programa
 - ❑ Si se exige que un programa lo calcule

Sistemas de tipos

- ❑ ¿Cómo establecer cuándo se cumple esta relación?
- ❑ Sistemas de reglas
- ❑ Si se permite que no haya un programa
 - ❑ Se sacrifica automatización para ganar precisión
 - ❑ Sistemas de tipos dependientes (COQ, Agda, etc.)
- ❑ Si se exige que un programa lo calcule
 - ❑ Se sacrifica precisión para ganar automatización
 - ❑ Vamos a estudiar éstos

Sistemas de tipos calculables



Visiones denotacional y operacional

❏ Visión denotacional

(significado)

❏ = (equivalencia de expresiones)

❏ Visión operacional

(mecánica, ejecutable)

❏ :: (tipado)

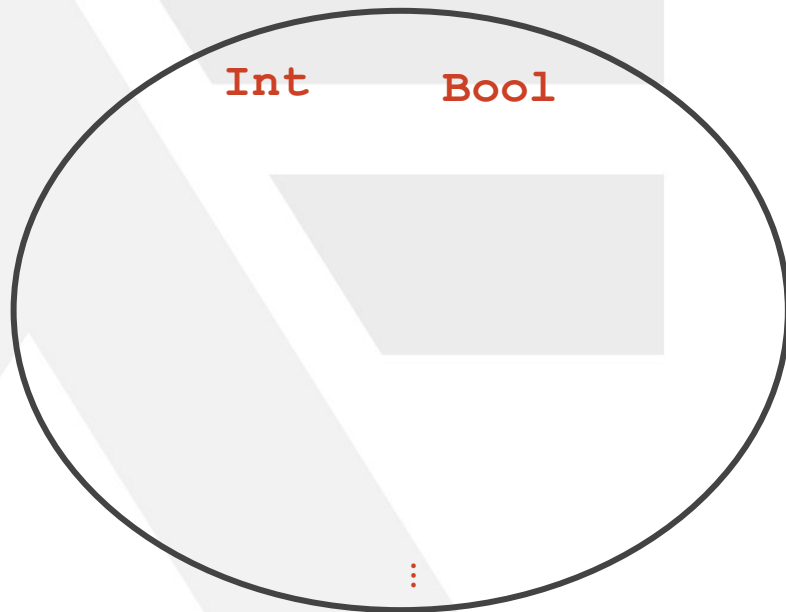
❏ → (reducción)

❏ Aceptamos una pérdida de precisión

❏ ¿pero cuánta? Depende de la definición del sistema

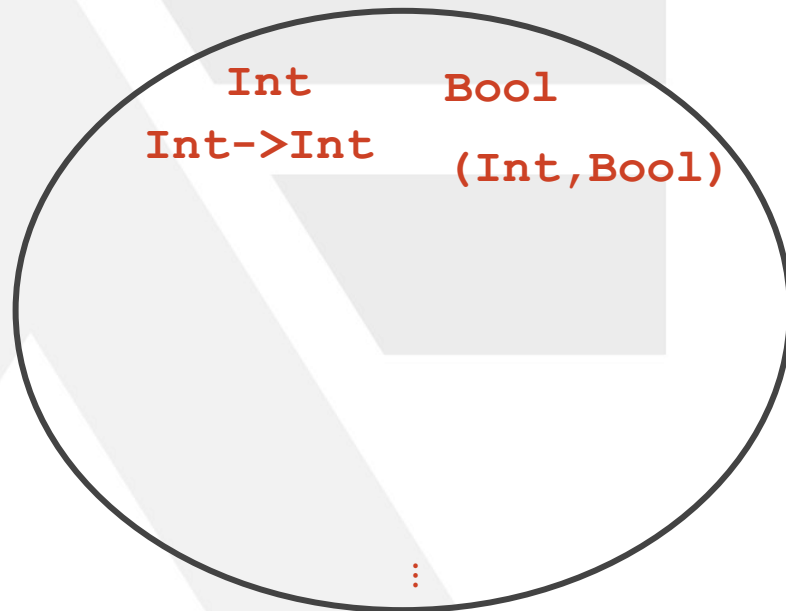
Sistema Hindley-Milner simple

- Se definen tipos (denominados genéricamente **A**, **B**)
 - Básicos
 - Int**, **Bool**, etc.
 - Compuestos
 - A** $\rightarrow$ **B**, **(A, B)**, etc.



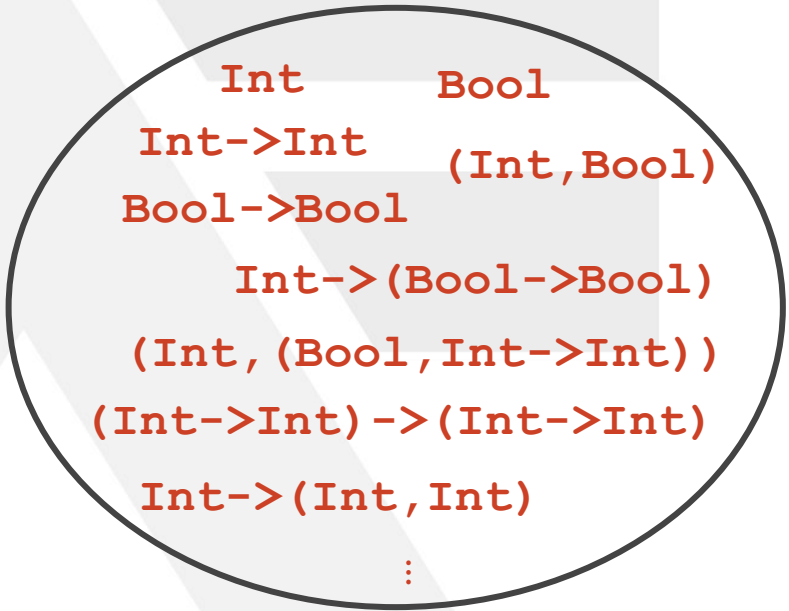
Sistema Hindley-Milner simple

- Se definen tipos (denominados genéricamente **A**, **B**)
 - Básicos
 - Int**, **Bool**, etc.
 - Compuestos
 - A $\rightarrow$ B**, **(A, B)**, etc.



Sistema Hindley-Milner simple

- Se definen tipos (denominados genéricamente **A**, **B**)
 - Básicos
 - Int**, **Bool**, etc.
 - Compuestos
 - A** $\rightarrow$ **B**, (**A**, **B**), etc.



Int **Bool**
Int $\rightarrow$ **Int** (**Int**, **Bool**)
Bool $\rightarrow$ **Bool**
 Int $\rightarrow$ (**Bool** $\rightarrow$ **Bool**)
 (**Int**, (**Bool**, **Int** $\rightarrow$ **Int**))
(**Int** $\rightarrow$ **Int**) $\rightarrow$ (**Int** $\rightarrow$ **Int**)
 Int $\rightarrow$ (**Int**, **Int**)
 :

Sistema Hindley-Milner simple

Se definen reglas para calcular la asignación

$n :: \text{Int}$

$\text{False} :: \text{Bool}$

Si $d = e$
y $e :: A$
entonces $d :: A$

$f :: A \rightarrow B$

$e :: A$

$f\ e :: B$

$e :: \text{Int}$

$e' :: \text{Int}$

$e + e' :: \text{Int}$

Sistema Hindley-Milner simple

- ❑ Con las reglas se puede calcular el tipo de una expresión a partir de su constitución

- ❑ La regla más importante

$$f :: A \rightarrow B$$
$$e :: A$$

$$f\ e :: B$$

- ❑ Dice que en una aplicación, la parte izquierda es una función y la parte derecha tiene el mismo tipo que el tipo del parámetro

Sistema Hindley-Milner simple

- Las reglas se leen como esquemas a encontrar

$$f :: A \rightarrow B$$
$$e :: A$$

$$f\ e :: B$$

Sistema Hindley-Milner simple

- Las reglas se leen como esquemas a encontrar

$$\frac{\begin{array}{l} \boxed{f} :: \boxed{A} \rightarrow \boxed{B} \\ \boxed{e} :: \boxed{A} \end{array}}{\boxed{f} \boxed{e} :: \boxed{B}}$$

Sistema Hindley-Milner simple

- Las reglas se leen como esquemas a encontrar

$$\frac{\begin{array}{c} f \quad \square :: \square^A \quad \text{---} \text{>} \quad \square^B \\ e \quad \square :: \square^A \end{array}}{\begin{array}{c} \square \quad \square :: \square^B \\ f \quad e \end{array}}$$

- Dice que en una aplicación, la parte izquierda es una función y la parte derecha tiene el mismo tipo que el tipo del parámetro

Robin Milner



Arthur John Robin Gorell Milner

(13 de enero 1934 – 20 de marzo 2010) es un científico de la computación británico, fundador del Laboratorio de Fundamentos de Ciencias de la Computación (LFCS) de la Universidad de Edimburgo.

Desarrolló la *Lógica para Funciones Computables* (**LCF**) en 1972, una de las primeras herramientas para demostración automatizada de teoremas, el lenguaje **ML** (Meta-Language) en 1973, primer lenguaje con inferencia de tipos polimórfica (el sistema de tipos Hindley-Milner), pensado para implementar la LCF, y el *Cálculo de Sistemas Comunicantes* (**CCS**) en 1980 y su sucesor, el **π -cálculo** en 1992, para analizar sistemas concurrentes.

Sistema H-M: ejemplo 1

Sistema Hindley-Milner simple

■ Ejemplo 1: $(\lambda x \rightarrow x+x) \ 2$

$(\lambda x \rightarrow x+x) \ 2 ::$

■ Se plantea la pregunta

Sistema Hindley-Milner simple

$f :: A \rightarrow B$
$e :: A$
$f\ e :: B$

■ Ejemplo 1: $(\backslash x \rightarrow x + x)\ 2$

$::$	$\rightarrow$
$::$	
$(\backslash x \rightarrow x + x)\ 2 ::$	

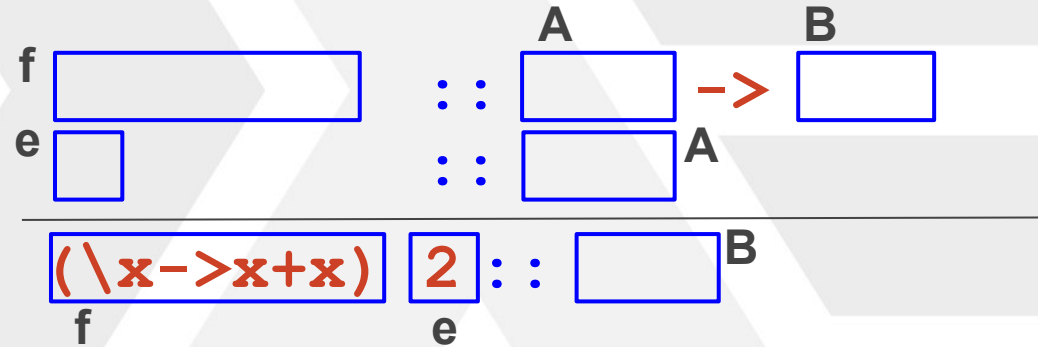
Porque se usó la
regla de aplicación

■ Se determina la regla a aplicar

Sistema Hindley-Milner simple

$f :: A \rightarrow B$
$e :: A$
$f\ e :: B$

■ Ejemplo 1: $(\lambda x \rightarrow x + x)\ 2$

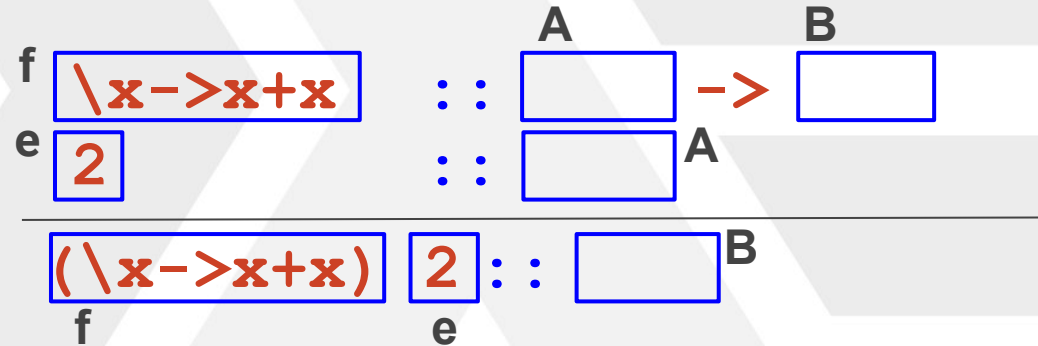


■ Se analizan cuáles son los componentes

$f :: A \rightarrow B$
$e :: A$
$f\ e :: B$

Sistema Hindley-Milner simple

■ Ejemplo 1: $(\lambda x \rightarrow x + x)\ 2$

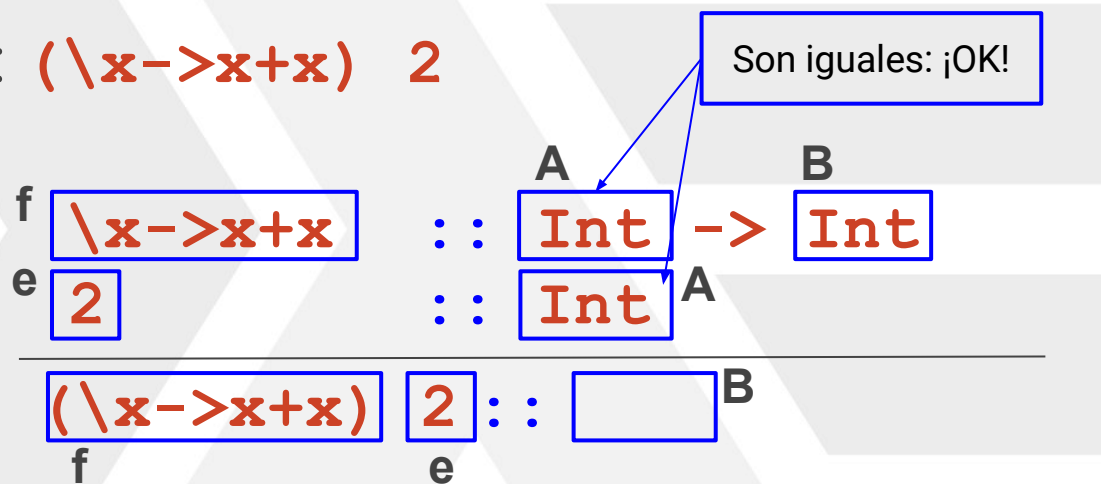


■ Se comienzan a colocar las piezas en su lugar

Sistema Hindley-Milner simple

$f :: A \rightarrow B$
$e :: A$
$f\ e :: B$

■ Ejemplo 1: $(\lambda x \rightarrow x+x)\ 2$



■ Se resuelven los antecedentes y se verifican las condiciones

Sistema Hindley-Milner simple

$f :: A \rightarrow B$
$e :: A$
$f\ e :: B$

❑ Ejemplo 1: $(\lambda x \rightarrow x + x)\ 2$

		A		B
f	$\lambda x \rightarrow x + x$	$::$	$\text{Int} \rightarrow$	Int
e	2	$::$	Int	A
	$(\lambda x \rightarrow x + x)\ 2$	$::$	Int	B
f			e	

❑ Se concluye con el resultado: tiene tipo Int

$(\lambda x \rightarrow x + x)\ 2 :: \text{Int}$

Sistema H-M: ejemplo 2

Sistema Hindley-Milner simple

❏ Ejemplo 2: **2 + False**

2 + False ::

❏ Se plantea la pregunta

Sistema Hindley-Milner simple

e	$:: \text{Int}$
e'	$:: \text{Int}$
$e + e'$	$:: \text{Int}$

■ Ejemplo 2: $2 + \text{False}$

$$\frac{\begin{array}{l} :: \text{Int} \\ :: \text{Int} \end{array}}{2 + \text{False} :: \text{Int}}$$

■ Se elige la regla a aplicar

Sistema Hindley-Milner simple

e	$:: \text{Int}$
e'	$:: \text{Int}$
$e + e'$	$:: \text{Int}$

❏ Ejemplo 2: $2 + \text{False}$

$$\frac{\begin{array}{l} e \quad \boxed{} \quad :: \boxed{\text{Int}}^A \\ e' \quad \boxed{\phantom{\text{False}}} \quad :: \boxed{\text{Int}}^A \end{array}}{\begin{array}{c} \boxed{2} + \boxed{\text{False}} :: \boxed{\text{Int}}^A \\ \underbrace{}_e \quad \underbrace{\phantom{\text{False}}}_{e'} \end{array}}$$

❏ Se analizan cuáles son los componentes

Sistema Hindley-Milner simple

e	$:: \text{Int}$
e'	$:: \text{Int}$
$e + e'$	$:: \text{Int}$

❑ Ejemplo 2: $2 + \text{False}$

$$\frac{\begin{array}{l} e \quad \boxed{2} \quad :: \boxed{\text{Int}}^A \\ e' \quad \boxed{\text{False}} \quad :: \boxed{\text{Int}}^A \end{array}}{\begin{array}{c} \boxed{2} + \boxed{\text{False}} :: \boxed{\text{Int}}^A \\ e \quad e' \end{array}}$$

❑ Se comienzan a colocar las piezas en su lugar

Sistema Hindley-Milner simple

e	$:: \text{Int}$
e'	$:: \text{Int}$
$e + e'$	$:: \text{Int}$

❑ Ejemplo 2: $2 + \text{False}$

$$\frac{\begin{array}{c} e \quad \boxed{2} \quad :: \quad \boxed{\text{Int}}^A \\ e' \quad \boxed{\text{False}} \quad :: \quad \boxed{\text{Int}}^A \end{array}}{\boxed{2} + \boxed{\text{False}} \quad :: \quad \boxed{\text{Int}}^A}$$

Note: In the original image, the second Int type in the antecedents is crossed out with a blue diagonal line.

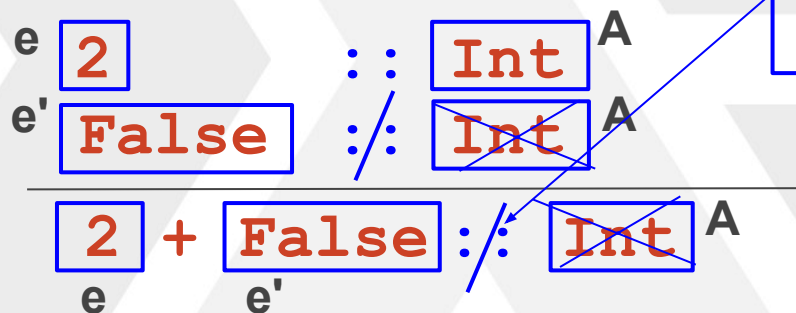
¡NO es cierto!
¡MAL!

❑ Se verifican las condiciones de los antecedentes

Sistema Hindley-Milner simple

e	$:: \text{Int}$
e'	$:: \text{Int}$
$e + e'$	$:: \text{Int}$

❑ Ejemplo 2: $2 + \text{False}$



¡NO puede haber un
antecedente falso!
¡MAL!

❑ Se concluye con el resultado: no es una expresión válida

$2 + \text{False} \quad /:$

Sistema H-M: ejemplo 3

Sistema Hindley-Milner simple

❏ Ejemplo 3: tipar definiciones `doblex = x+x`

`doblex ::`

❏ Se plantea la pregunta

Sistema Hindley-Milner simple

❏ Ejemplo 3: tipar definiciones `doble x = x+x`

`doble ::`

`doble x ::`

`x + x ::`

❏ Se desarrollan las partes de la definición

Sistema Hindley-Milner simple

f	::	A	->	B
e	::	A		
f e	::	B		

e	::	Int
e'	::	Int
e + e'	::	Int

■ Ejemplo 3: tipar definiciones **doble x = x+x**

doble	::		->	
	::			
doble x	::			

Porque se usó la
regla de aplicación

	::	Int
	::	Int
x + x	::	Int

Porque se usó la
regla de la suma

■ Se determinan las reglas a aplicar

Sistema Hindley-Milner simple

f	:: A → B
e	:: A
f e	:: B

e	:: Int
e'	:: Int
e + e'	:: Int

❑ Ejemplo 3: tipar definiciones **doble x = x+x**

f	doble	::		→	
e		::		A	
f	doble	x	::		B

e'		::	Int	A'
e''		::	Int	A'
x	+	x	::	Int A'
e'		e''		

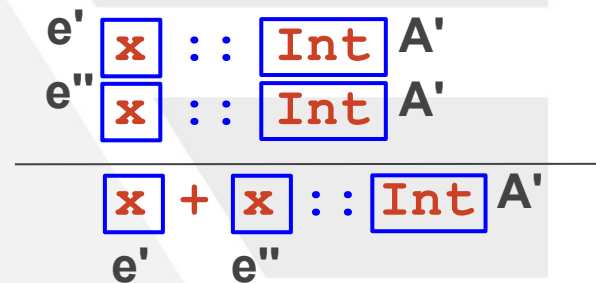
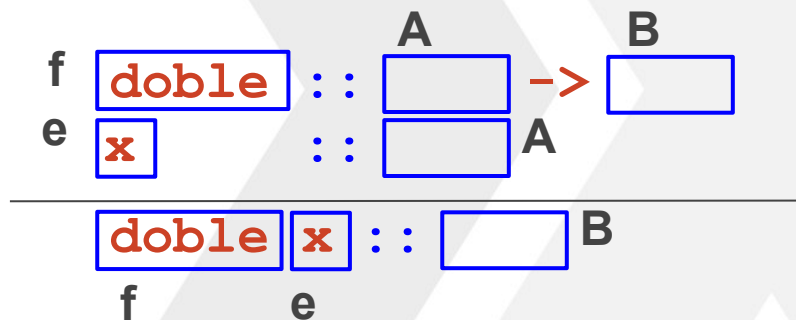
❑ Se analizan cuáles son los componentes

Sistema Hindley-Milner simple

f	:: A -> B
e	:: A
f e	:: B

e	:: Int
e'	:: Int
e + e'	:: Int

❑ Ejemplo 3: tipar definiciones **doble x = x+x**



❑ Se comienzan a colocar las piezas en su lugar

Sistema Hindley-Milner simple

f	:: A $\rightarrow$ B
e	:: A
f e	:: B

e	:: Int
e'	:: Int
e + e'	:: Int

❑ Ejemplo 3: tipar definiciones **doble x = x+x**

f	doble	::		$\rightarrow$	
e	x	::			
			A		B
f	doble	x	::		B

e'	x	::	Int	A'
e''	x	::	Int	A'
	x	+	x	:: Int A'
e'			e''	

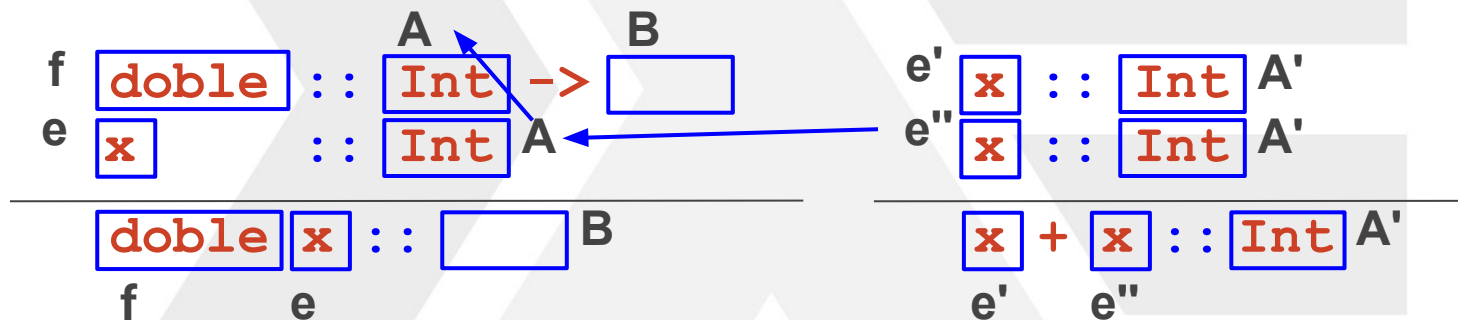
Son iguales:
¡OK!

❑ Se verifican las condiciones

Sistema Hindley-Milner simple

f	$:: A \rightarrow B$	e	$:: \text{Int}$
e	$:: A$	e'	$:: \text{Int}$
$f e :: B$		$e + e' :: \text{Int}$	

❑ Ejemplo 3: tipar definiciones **doble x = x+x**

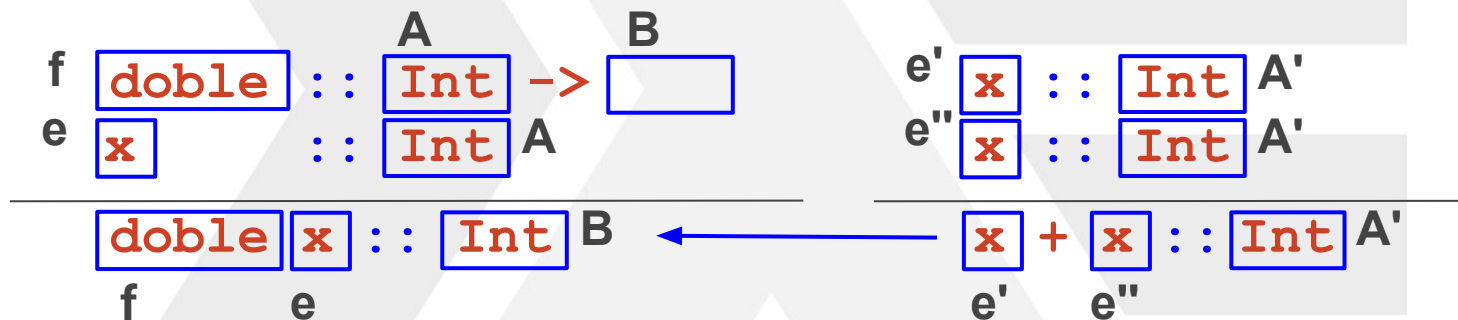


❑ Se propaga la información (es la misma **x** en todos lados)

Sistema Hindley-Milner simple

Si $d = e$
 y $e :: A$
 entonces $d :: A$

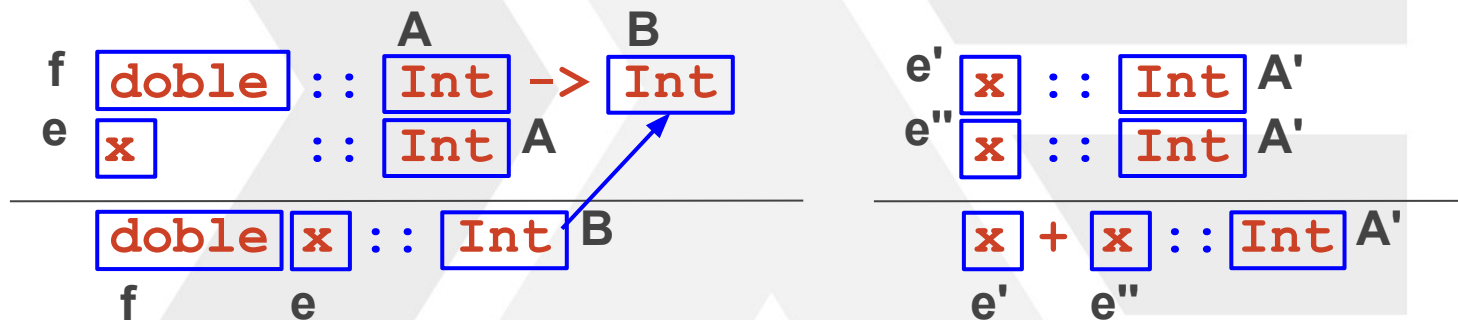
□ Ejemplo 3: tipar definiciones $\text{doble } x = x + x$



□ Se propaga la información (con la regla de tipar definiciones)

Sistema Hindley-Milner simple

□ Ejemplo 3: tipar definiciones **doblex = x+x**



□ Se concluye con el resultado: función de números en números

doblex :: Int -> Int

Sistema Hindley-Milner simple

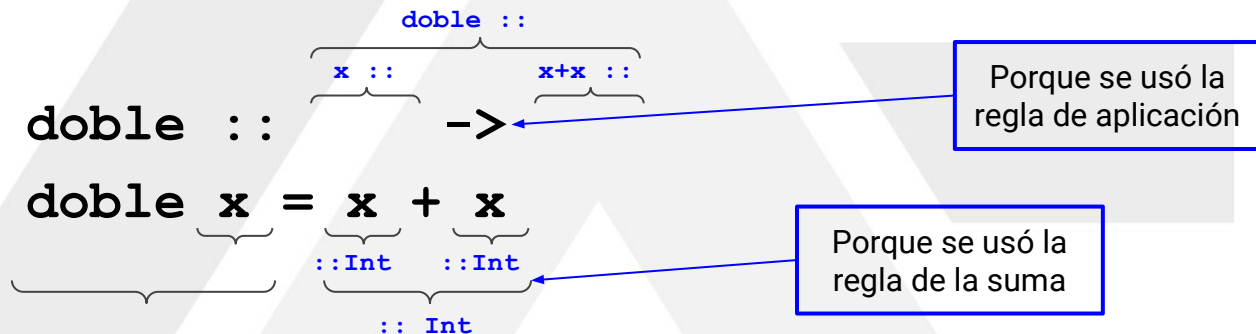
- ❑ Ejemplo 3: tipar definiciones **`doble x = x+x`**
- ❑ Se puede lograr lo mismo sin expandir las reglas

```
doble ::  
doble x = x + x
```

- ❑ Se plantea la pregunta

Sistema Hindley-Milner simple

- ❑ Ejemplo 3: tipar definiciones **doble x = x+x**
- ❑ Se puede lograr lo mismo sin expandir las reglas



- ❑ Se analizan las partes y se determinan las reglas a aplicar

Sistema Hindley-Milner simple

- ❑ Ejemplo 3: tipar definiciones **doble x = x+x**
- ❑ Se puede lograr lo mismo sin expandir las reglas

$$\begin{array}{c} \text{doble} :: \\ \text{doble} :: \overbrace{\text{Int} \rightarrow \text{Int}}^{x :: \text{Int}, x+x :: \text{Int}} \\ \text{doble } x = \underbrace{x}_{:: \text{Int}} + \underbrace{x}_{:: \text{Int}} \\ \underbrace{\quad}_{:: \text{Int}} \quad \underbrace{\quad}_{:: \text{Int}} \end{array}$$

- ❑ Se propaga la información

Sistema Hindley-Milner simple

- ❑ Ejemplo 3: tipar definiciones **doble x = x+x**
- ❑ Se puede lograr lo mismo sin expandir las reglas

```
doble :: Int -> Int  
doble x = x + x
```

- ❑ Se concluye con el resultado

Sistema H-M: ejemplo 4

Sistema Hindley-Milner simple

■ Ejemplo 4: **`twice f = g where g x = f (f x)`**

`twice ::`

■ Se plantea la pregunta

Sistema Hindley-Milner simple

■ Ejemplo 4: **`twice f = g where g x = f (f x)`**

`twice ::`

`twice f ::`

`g ::`

`g x ::`

`f (f x) ::`

`f x ::`

■ Se desarrollan las partes de la definición

Sistema Hindley-Milner simple

f	::	A	->	B
e	::	A		
f e	::	B		

■ Ejemplo 4: **twice** $f = g$ where $g\ x = f\ (f\ x)$

twice ::
 ->
 ::

twice f ::

g ::
 ->
 ::

g x ::

 ::
 ->
 ::

f (f x) ::

 ::
 ->
 ::

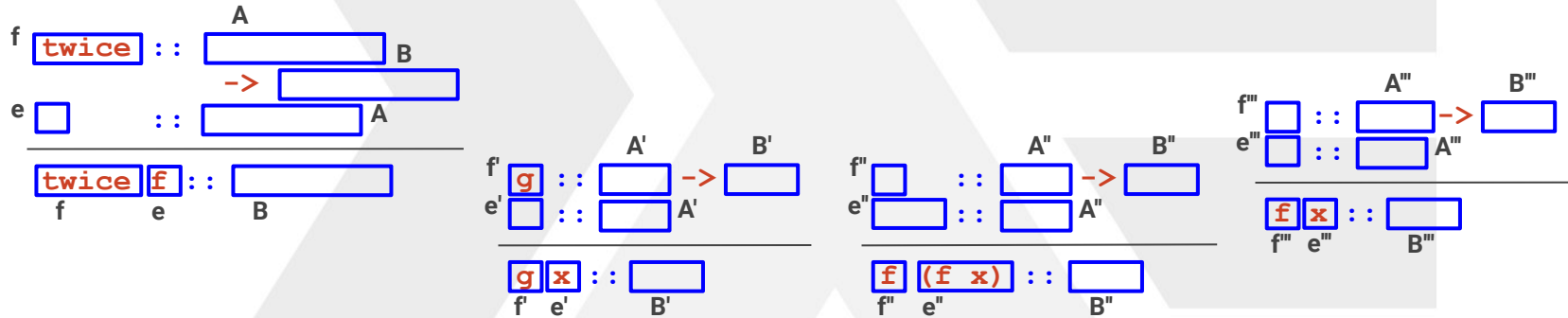
f x ::

■ Se determinan las reglas a aplicar

Sistema Hindley-Milner simple

f	$:: A \rightarrow B$
e	$:: A$
$f\ e$	$:: B$

■ Ejemplo 4: **twice** $f = g$ where $g\ x = f\ (f\ x)$

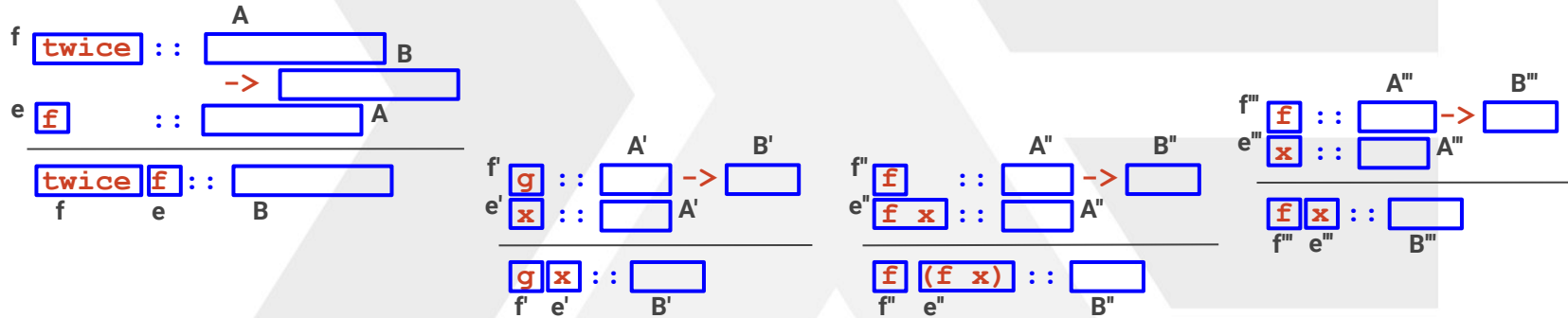


■ Se analizan cuáles son los componentes

Sistema Hindley-Milner simple

f	::	A → B
e	::	A
f e	::	B

■ Ejemplo 4: **twice** $f = g$ where $g\ x = f\ (f\ x)$

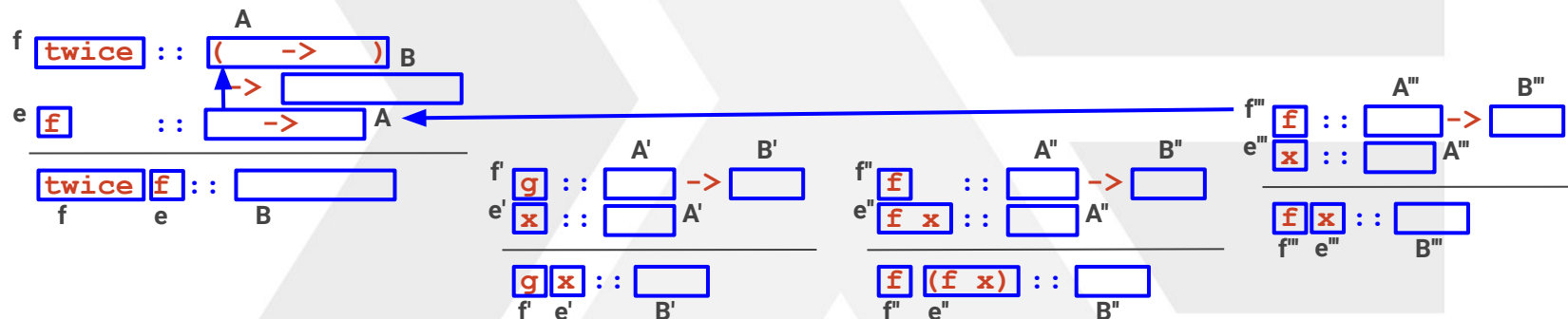


■ Se comienzan a colocar las piezas en su lugar

Sistema Hindley-Milner simple

f	$:: A \rightarrow B$
e	$:: A$
$f\ e$	$:: B$

■ Ejemplo 4: **twice** $f = g$ where $g\ x = f\ (f\ x)$

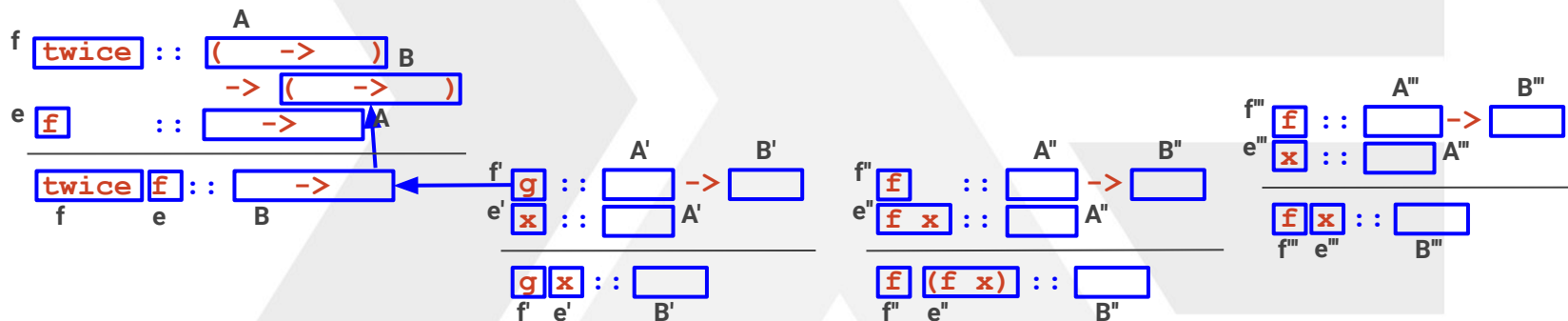


■ Se propaga la información (es la misma **f** en todos lados)

Sistema Hindley-Milner simple

f	$:: A \rightarrow B$
e	$:: A$
$f\ e$	$:: B$

❑ Ejemplo 4: **twice** $f = g$ where $g\ x = f\ (f\ x)$

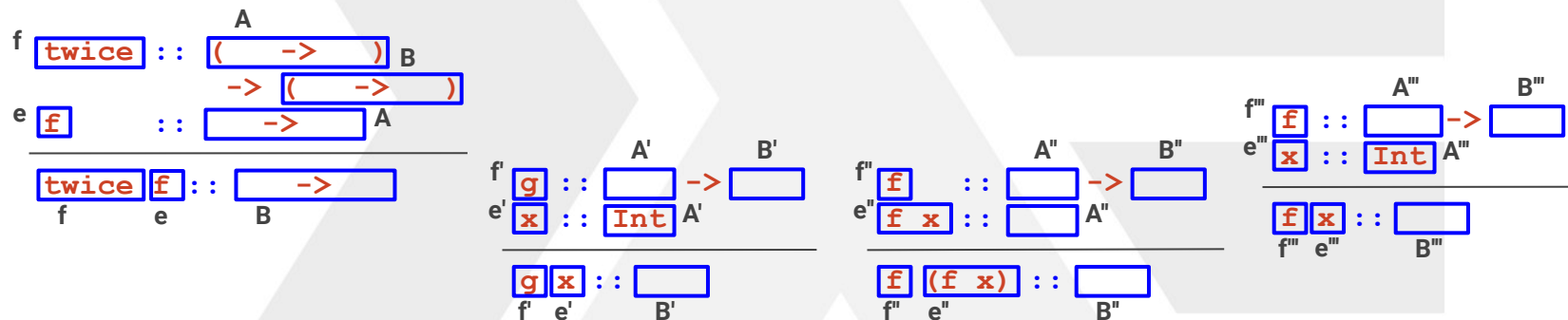


❑ Se propaga la información (con la regla de tipar definiciones)

Sistema Hindley-Milner simple

f	$:: A \rightarrow B$
e	$:: A$
$f\ e$	$:: B$

❑ Ejemplo 4: **twice** $f = g$ where $g\ x = f\ (f\ x)$

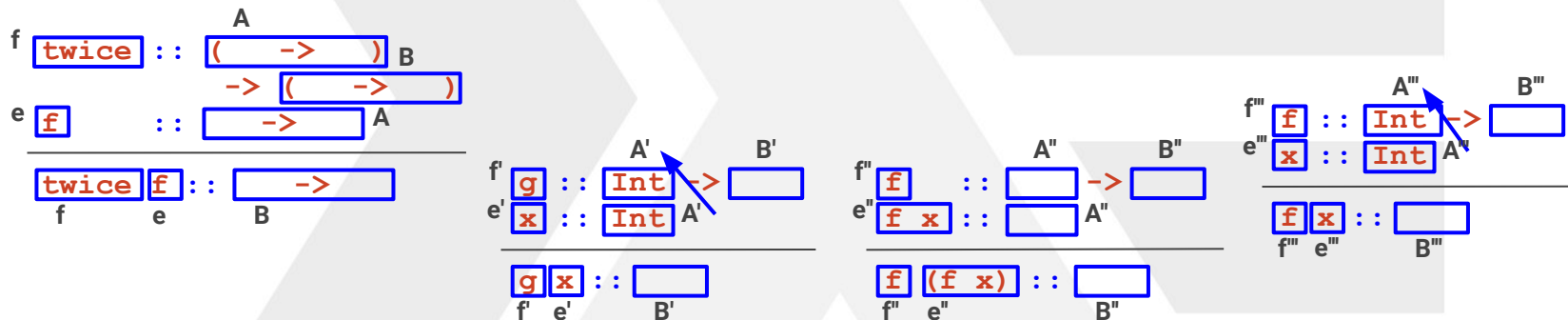


❑ Se decide el tipo de **x**: por ejemplo, **Int**
(porque no hay restricciones)

Sistema Hindley-Milner simple

f	$:: A \rightarrow B$
e	$:: A$
$f\ e$	$:: B$

❑ Ejemplo 4: **twice** $f = g$ where $g\ x = f\ (f\ x)$

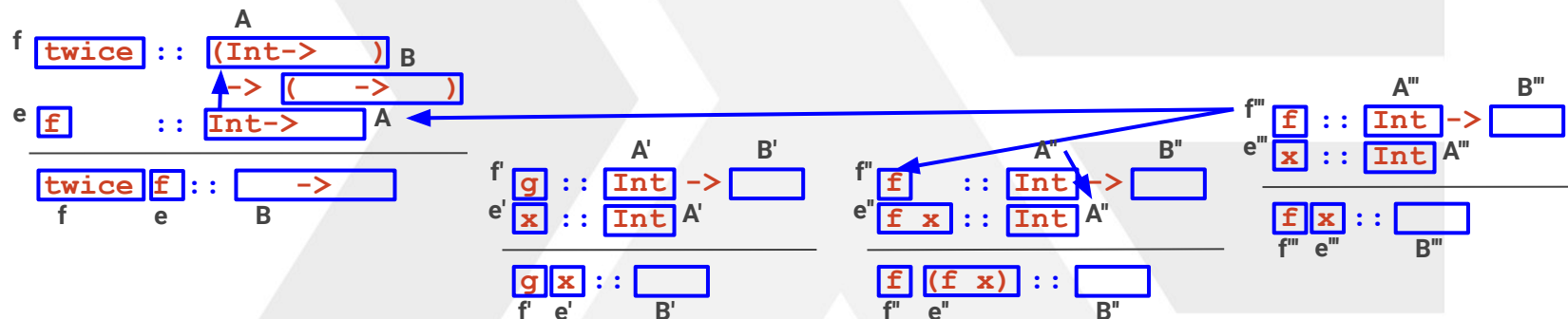


❑ Se propaga la información (por igualdad de componentes)

Sistema Hindley-Milner simple

f	$:: A \rightarrow B$
e	$:: A$
$f\ e$	$:: B$

❑ Ejemplo 4: **twice** $f = g$ where $g\ x = f\ (f\ x)$

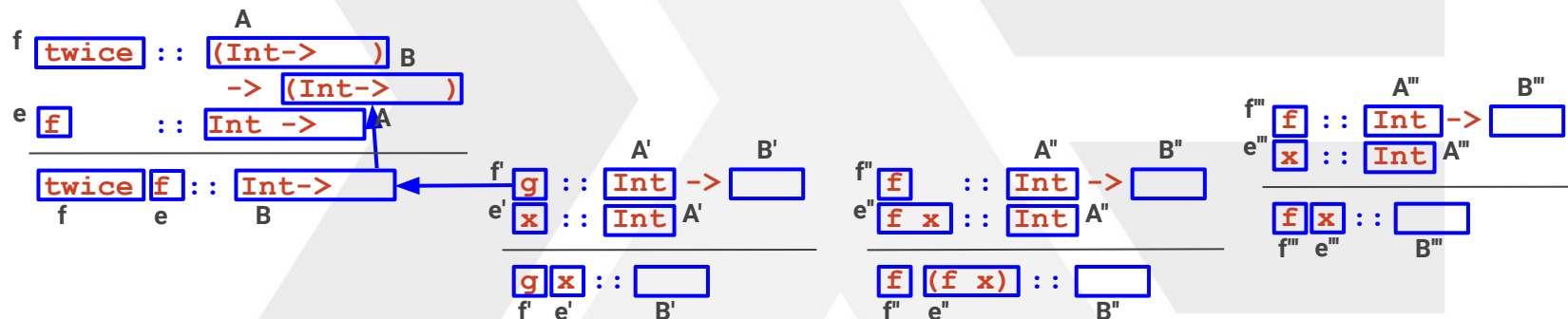


❑ Se propaga la información (es la misma **f** en todos lados)

Sistema Hindley-Milner simple

f	$:: A \rightarrow B$
e	$:: A$
$f\ e$	$:: B$

■ Ejemplo 4: `twice f = g` where `g x = f (f x)`

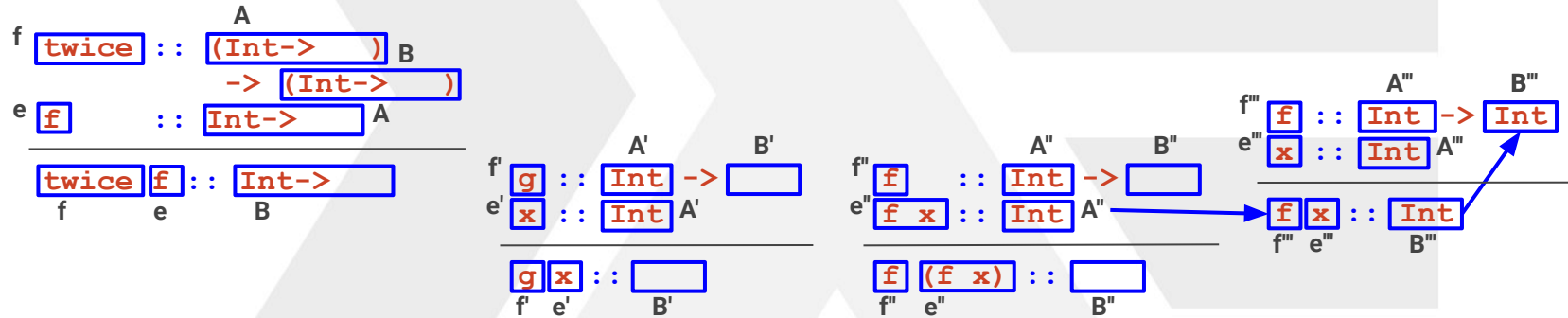


■ Se propaga la información (con la regla de tipar definiciones)

Sistema Hindley-Milner simple

f	$:: A \rightarrow B$
e	$:: A$
$f\ e$	$:: B$

❑ Ejemplo 4: **twice** $f = g$ where $g\ x = f\ (f\ x)$

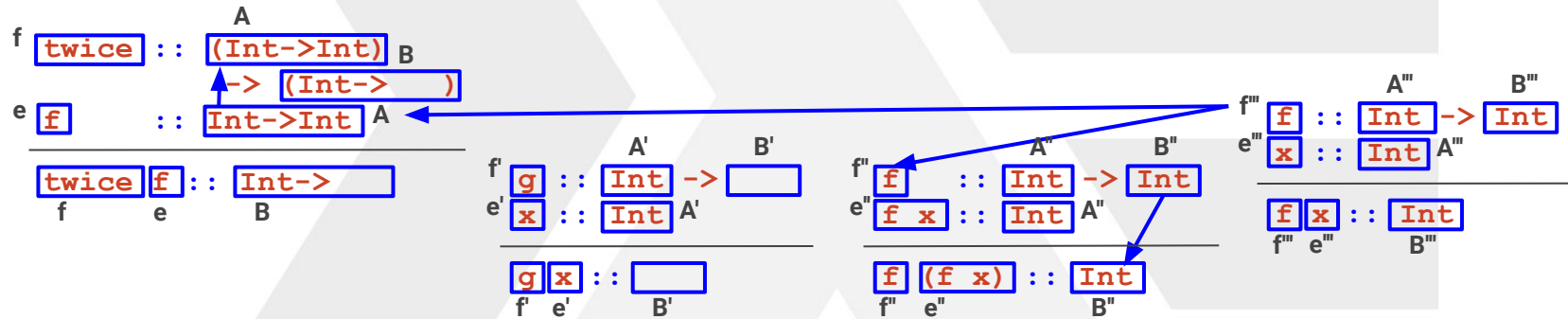


❑ Se propaga la información (**f x** es la misma ambas veces)

Sistema Hindley-Milner simple

f	$:: A \rightarrow B$
e	$:: A$
$f\ e$	$:: B$

❑ Ejemplo 4: **twice** $f = g$ where $g\ x = f\ (f\ x)$

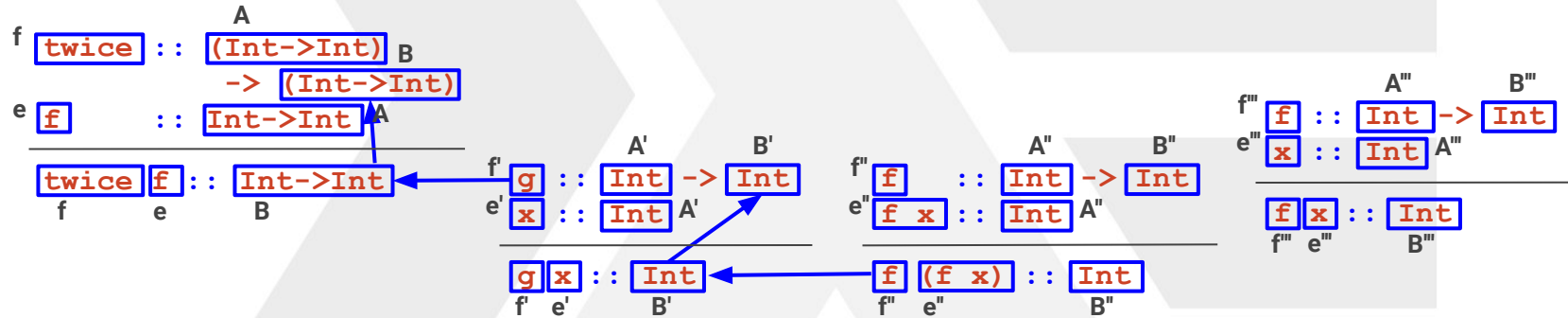


❑ Se propaga la información (es la misma f en todos lados)

Sistema Hindley-Milner simple

f	$:: A \rightarrow B$
e	$:: A$
$f\ e$	$:: B$

❑ Ejemplo 4: **twice** $f = g$ where $g\ x = f\ (f\ x)$

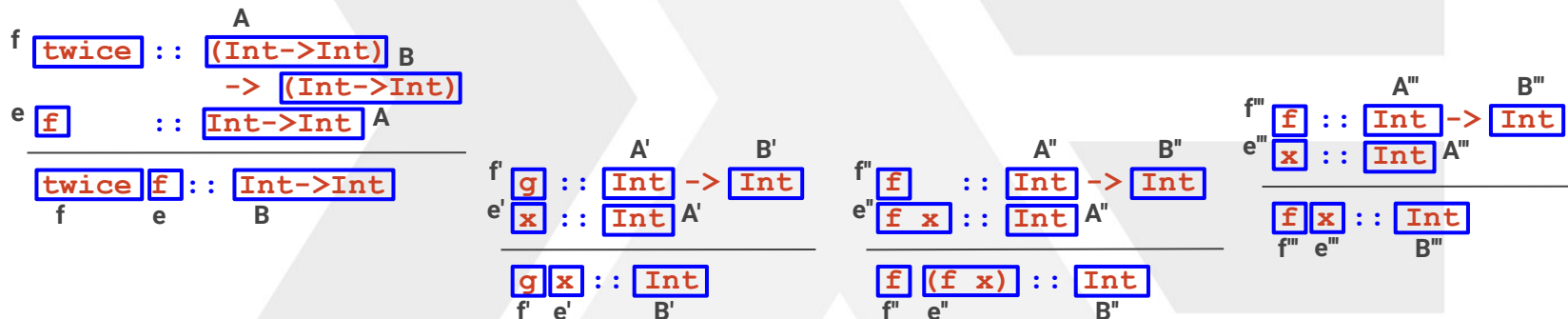


❑ Se propaga la información (con la regla de tipar definiciones)

Sistema Hindley-Milner simple

f	$:: A \rightarrow B$
e	$:: A$
$f\ e$	$:: B$

❑ Ejemplo 4: **twice** $f = g$ where $g\ x = f\ (f\ x)$



❑ Se concluye con el resultado: función de números en números

twice $:: (Int \rightarrow Int) \rightarrow (Int \rightarrow Int)$

Sistema Hindley-Milner simple

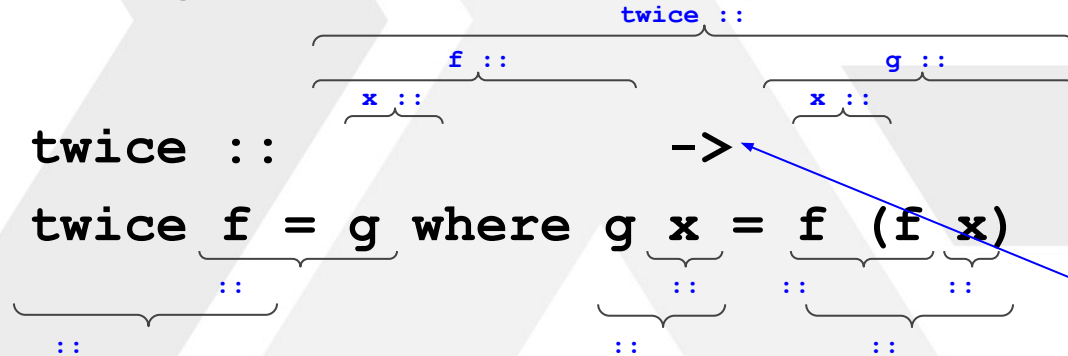
- ❑ Ejemplo 4: **`twice f = g where g x = f (f x)`**
- ❑ Se puede lograr lo mismo sin expandir las reglas

```
twice ::  
twice f = g where g x = f (f x)
```

- ❑ Se plantea la pregunta

Sistema Hindley-Milner simple

- ❑ Ejemplo 4: `twice f = g where g x = f (f x)`
- ❑ Se puede lograr lo mismo sin expandir las reglas



Porque se usó la
regla de aplicación

- ❑ Se analizan las partes y se determinan las reglas a aplicar

Sistema Hindley-Milner simple

- ❑ Ejemplo 4: **twice** $f = g$ where $g\ x = f\ (f\ x)$
- ❑ Se puede lograr lo mismo sin expandir las reglas

$$\text{twice} :: \left(\overbrace{\left(\overbrace{x ::}^{f ::} \rightarrow \right)}^{g ::} \rightarrow \right)$$
$$\text{twice } f = g \text{ where } g\ x = f\ (f\ x)$$

Diagram illustrating the typing of the `twice` function. The first line shows the type signature `twice :: ( (x :: f ::) -> ) -> ( (x :: g ::) -> )` with annotations. The second line shows the definition `twice f = g where g x = f (f x)` with annotations. Blue arrows point from the `f` in the definition to the `f ::` in the signature, and from the `f (f x)` to the `g ::` in the signature.

Porque se usó la
regla de aplicación

- ❑ Se deciden más reglas y se propaga la información

Sistema Hindley-Milner simple

- ❑ Ejemplo 4: **twice** $f = g$ where $g\ x = f\ (f\ x)$
- ❑ Se puede lograr lo mismo sin expandir las reglas

$$\begin{array}{c}
 \text{twice} :: \overbrace{\left(\overbrace{\left(\overbrace{x ::}^{\text{f} ::} \rightarrow \right)}^{\text{g} ::} \rightarrow \right)}^{\text{twice} ::} \\
 \text{twice } f = g \text{ where } g\ x = f\ (f\ x) \\
 \underbrace{\text{twice}}_{::} \underbrace{f = g}_{:: \rightarrow} \text{ where } \underbrace{g\ x}_{:: \text{Int}} = \underbrace{f}_{::} \underbrace{(f\ x)}_{:: \rightarrow :: \text{Int}}
 \end{array}$$

- ❑ Se elige un tipo para x (p.ej., **Int**, porque no hay restricciones)

Sistema Hindley-Milner simple

- ❑ Ejemplo 4: **twice f = g where g x = f (f x)**
- ❑ Se puede lograr lo mismo sin expandir las reglas

$$\begin{array}{c}
 \text{twice} :: \overbrace{(\overbrace{\text{Int} \rightarrow}^{f ::} \text{ }) \rightarrow (\overbrace{\text{Int} \rightarrow}^{g ::} \text{ })} \\
 \text{twice } f = g \text{ where } g \, x = f \, (f \, x) \\
 \underbrace{\underbrace{\text{twice } f}_{:: \text{Int} \rightarrow} = \underbrace{g}_{:: \text{Int}}}_{:: \text{Int} \rightarrow} \, x = \underbrace{f}_{:: \text{Int} \rightarrow} \, (\underbrace{f}_{:: \text{Int}} \, \underbrace{x}_{:: \text{Int}})
 \end{array}$$

- ❑ Se propaga la información (a todos los lugares donde está **x**)

Sistema Hindley-Milner simple

- ❑ Ejemplo 4: **twice f = g where g x = f (f x)**
- ❑ Se puede lograr lo mismo sin expandir las reglas

$$\begin{array}{c}
 \text{twice} :: \overbrace{(\overbrace{\text{Int} \rightarrow}^{f ::} \text{ }) \rightarrow (\overbrace{\text{Int} \rightarrow}^{g ::} \text{ })}^{\text{twice} ::} \\
 \text{twice } \underbrace{f = g \text{ where } g \text{ } x = f (f \text{ } x)}_{\substack{\text{:: Int} \rightarrow \\ \text{:: Int} \quad \text{:: Int} \rightarrow \text{Int} \text{ :: Int} \\ \text{::} \quad \quad \quad \text{::}}}
 \end{array}$$

- ❑ Se propaga la información (porque **(f x)** es argumento de **f**)

Sistema Hindley-Milner simple

- ❑ Ejemplo 4: **twice f = g where g x = f (f x)**
- ❑ Se puede lograr lo mismo sin expandir las reglas

$$\begin{array}{c}
 \text{twice} :: \overbrace{(\overbrace{\text{Int} \rightarrow \text{Int}}^{f :: \text{Int} \rightarrow \text{Int}} \rightarrow \overbrace{\text{Int} \rightarrow \text{Int}}^{g :: \text{Int} \rightarrow \text{Int}})}^{\text{twice} :: \text{Int} \rightarrow \text{Int}} \\
 \text{twice } f = g \text{ where } g \text{ } x = f (f \text{ } x) \\
 \underbrace{\text{twice } f}_{:: \text{Int} \rightarrow \text{Int}} = \underbrace{g}_{:: \text{Int} \rightarrow \text{Int}} \text{ } \underbrace{x}_{:: \text{Int}} = \underbrace{f}_{:: \text{Int} \rightarrow \text{Int}} (\underbrace{f}_{:: \text{Int} \rightarrow \text{Int}} \underbrace{x}_{:: \text{Int}})
 \end{array}$$

- ❑ Se propaga la información (por igualdad de componentes)

Sistema Hindley-Milner simple

- ❑ Ejemplo 4: **twice f = g where g x = f (f x)**
- ❑ Se puede lograr lo mismo sin expandir las reglas

$$\begin{array}{c}
 \text{twice} :: \overbrace{(\overbrace{\text{Int} \rightarrow}^{f ::} \text{ }) \rightarrow (\overbrace{\text{Int} \rightarrow}^{g ::} \text{ })}^{\text{twice} ::} \\
 \text{twice } \underbrace{f = g}_{:: \text{Int} \rightarrow} \text{ where } \underbrace{g \ x}_{:: \text{Int}} = \underbrace{f}_{:: \text{Int} \rightarrow \text{Int} :: \text{Int}} (\underbrace{f \ x}_{:: \text{Int}}) \\
 \underbrace{\quad}_{:: \text{Int} \rightarrow} \qquad \qquad \qquad \underbrace{\quad}_{:: \text{Int}} \qquad \qquad \underbrace{\quad}_{:: \text{Int}}
 \end{array}$$

- ❑ Se propaga la información (con la regla de tipar definiciones)

Sistema Hindley-Milner simple

- ❑ Ejemplo 4: **twice f = g where g x = f (f x)**
- ❑ Se puede lograr lo mismo sin expandir las reglas

$$\begin{array}{c} \text{twice} :: \underbrace{\underbrace{\underbrace{x :: \text{Int}}_{f :: \text{Int} \rightarrow \text{Int}}}_{\text{twice} :: \text{Int} \rightarrow \text{Int}}} \rightarrow \underbrace{\underbrace{\underbrace{x :: \text{Int}}_{g :: \text{Int} \rightarrow \text{Int}}}_{\text{twice f = g where g x = f (f x)}}_{\text{twice f = g where g x = f (f x)}} \\ \text{twice} :: (\text{Int} \rightarrow \text{Int}) \rightarrow (\text{Int} \rightarrow \text{Int}) \\ \text{twice f = g where g x = f (f x)} \\ \underbrace{\underbrace{\text{twice f = g where g x = f (f x)}}_{:: \text{Int} \rightarrow \text{Int}}}_{:: \text{Int} \rightarrow \text{Int}} \quad \underbrace{\underbrace{\text{g x}}_{:: \text{Int}}}_{:: \text{Int}} \quad \underbrace{\underbrace{\text{f (f x)}}_{:: \text{Int} \rightarrow \text{Int} :: \text{Int}}}_{:: \text{Int}} \end{array}$$

- ❑ Se propaga la información (por igualdad de componentes)

Sistema Hindley-Milner simple

- ❑ Ejemplo 4: **`twice f = g where g x = f (f x)`**
- ❑ Se puede lograr lo mismo sin expandir las reglas

```
twice :: (Int->Int) -> (Int->Int)  
twice f = g where g x = f (f x)
```

- ❑ Se concluye con la respuesta



Polimorfismo

Polimorfismo: motivación

❑ Ejemplo 5: (twice doble, twice not)

twice :: (Int->Int) -> (Int->Int)

f	twice	::	$(\text{Int} \rightarrow \text{Int})$	B
			$\rightarrow (\text{Int} \rightarrow \text{Int})$	
e	doble	::	$\text{Int} \rightarrow \text{Int}$	A
f	twice	doble	::	$\text{Int} \rightarrow \text{Int}$ B
		e		

f'	twice	::	$(\text{Int} \rightarrow \text{Int})$	B'
			$\rightarrow (\text{Int} \rightarrow \text{Int})$	
e'	not	::	$\text{Bool} \rightarrow \text{Bool}$	A'
f'	twice	not	::	$\text{Int} \rightarrow \text{Int}$ A'
		e'		

NO son iguales:
¡MAL!

❑ ¿Por qué no se pueden darle tipo a ambas partes?

Polimorfismo: motivación

❑ Ejemplo 5: (`twice doble`, `twice not`)

`twice :: (Int->Int) -> (Int->Int)`

NO son iguales:
¡MAL!

$$\begin{array}{c} \text{:: (Int->Int)} \longleftrightarrow \text{(Int->Int)} \\ \swarrow \quad \searrow \\ \underbrace{(\text{twice doble})}_{\text{:: Int->Int}} \quad \underbrace{(\text{twice not})}_{\text{:: Bool->Bool}} \end{array} \quad \text{twice doble :: twice not ::}$$

$$(\text{twice doble}, \text{twice not}) \text{ :: } (\text{Int->Int}, \text{X})$$

❑ ¿Por qué no se pueden darle tipo a ambas partes?

Polimorfismo: motivación

- ❑ Ejemplo 5: (**twice** **doble**, **twice** **not**)
twice :: (Int->Int) -> (Int->Int)
- ❑ ¿Por qué no se pueden darle tipo a ambas partes?
 - ❑ En el ej.4 el tipo de **x** fue elegido como **Int**
 - ❑ Pero entonces **not** no puede ser argumento de **twice**
 - ❑ ¿Podría elegirse otra cosa?

Polimorfismo: motivación

■ Ejemplo 4, revisado: **twice**

■ (partiendo desde el punto de elección del tipo de **x**)

$$\begin{array}{c} \text{twice} :: \overbrace{\left(\overbrace{\left(\overbrace{x ::}^f :: \right) \rightarrow}^g :: \right) \rightarrow} \\ \text{twice } f = g \text{ where } g \text{ } x = f \text{ } (f \text{ } x) \\ \underbrace{\quad \quad \quad} :: \rightarrow \quad \quad \quad \underbrace{\quad \quad} :: \text{Bool} \quad \underbrace{\quad \quad} :: \rightarrow \quad \underbrace{\quad \quad} :: \text{Bool} \\ \underbrace{\quad \quad} :: \rightarrow \quad \quad \quad \underbrace{\quad \quad} :: \quad \quad \quad \underbrace{\quad \quad} :: \end{array}$$

■ Se elige **otro** tipo para **x** (p.ej., **Bool**, porque no hay restricciones)

Polimorfismo: motivación

■ Ejemplo 4, revisado: **twice**

■ (partiendo desde el punto de elección del tipo de **x**)

$$\begin{array}{c} \text{twice} :: \overbrace{\overbrace{f :: (Bool \rightarrow Bool)}^{x ::} \rightarrow \overbrace{g :: (Bool \rightarrow Bool)}^{x ::}} \\ \text{twice } f = g \text{ where } g \text{ } x = f \text{ (} f \text{ } x \text{)} \\ \underbrace{\underbrace{\underbrace{:: Bool \rightarrow Bool} \quad \underbrace{:: Bool \rightarrow Bool} \quad \underbrace{:: Bool \rightarrow Bool}}_{:: Bool \rightarrow Bool} \quad \underbrace{\underbrace{:: Bool} \quad \underbrace{:: Bool} \quad \underbrace{:: Bool}}_{:: Bool} \quad \underbrace{\underbrace{::} \quad \underbrace{->} \quad \underbrace{:: Bool}}_{:: Bool} \end{array}$$

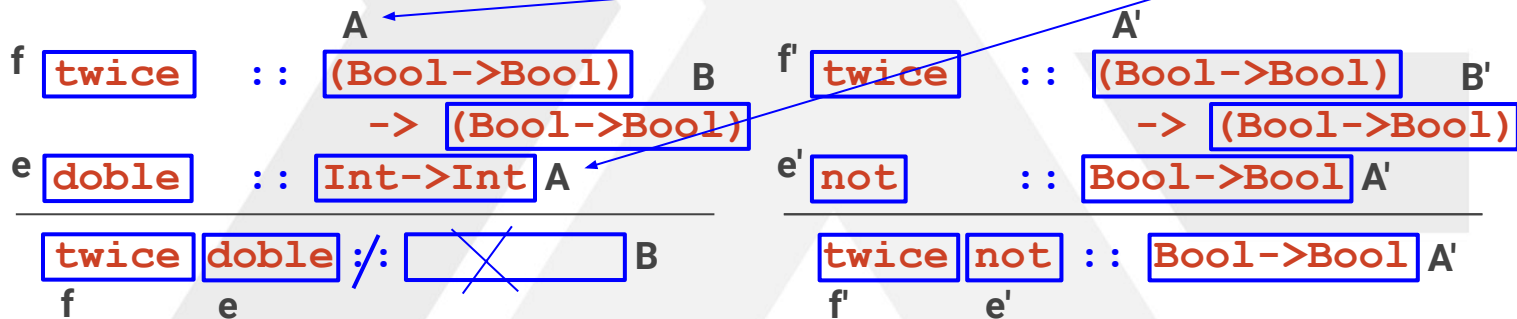
■ Y se propaga la información

Polimorfismo: motivación

❑ Ejemplo 5: (twice doble, twice not)

`twice :: (Bool->Bool) -> (Bool->Bool)`

NO son iguales:
¡MAL!



❑ ¿Por qué no se pueden darle tipo a ambas partes?

Polimorfismo: motivación

❏ Ejemplo 5, revisado

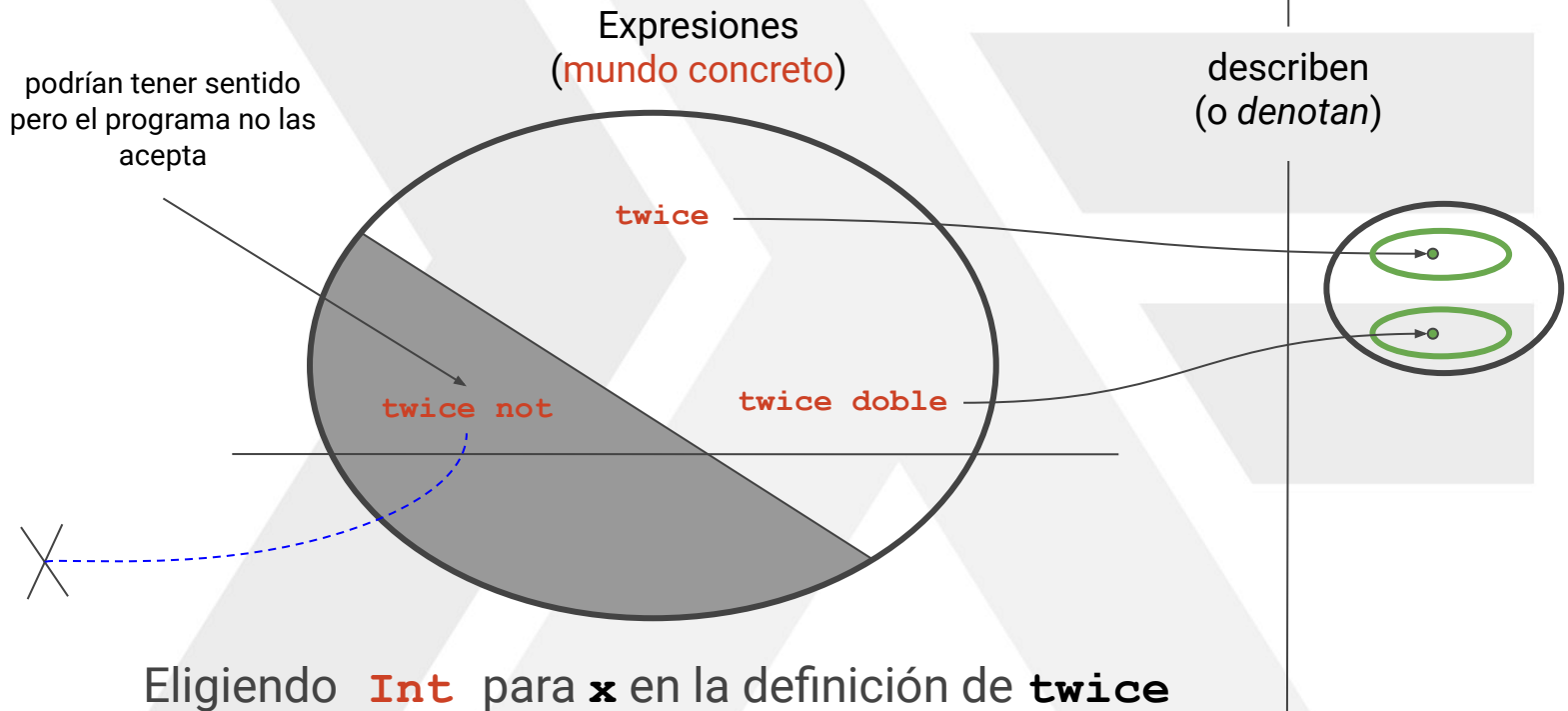
twice :: (Bool->Bool) -> (Bool->Bool)

NO son iguales:
¡MAL!

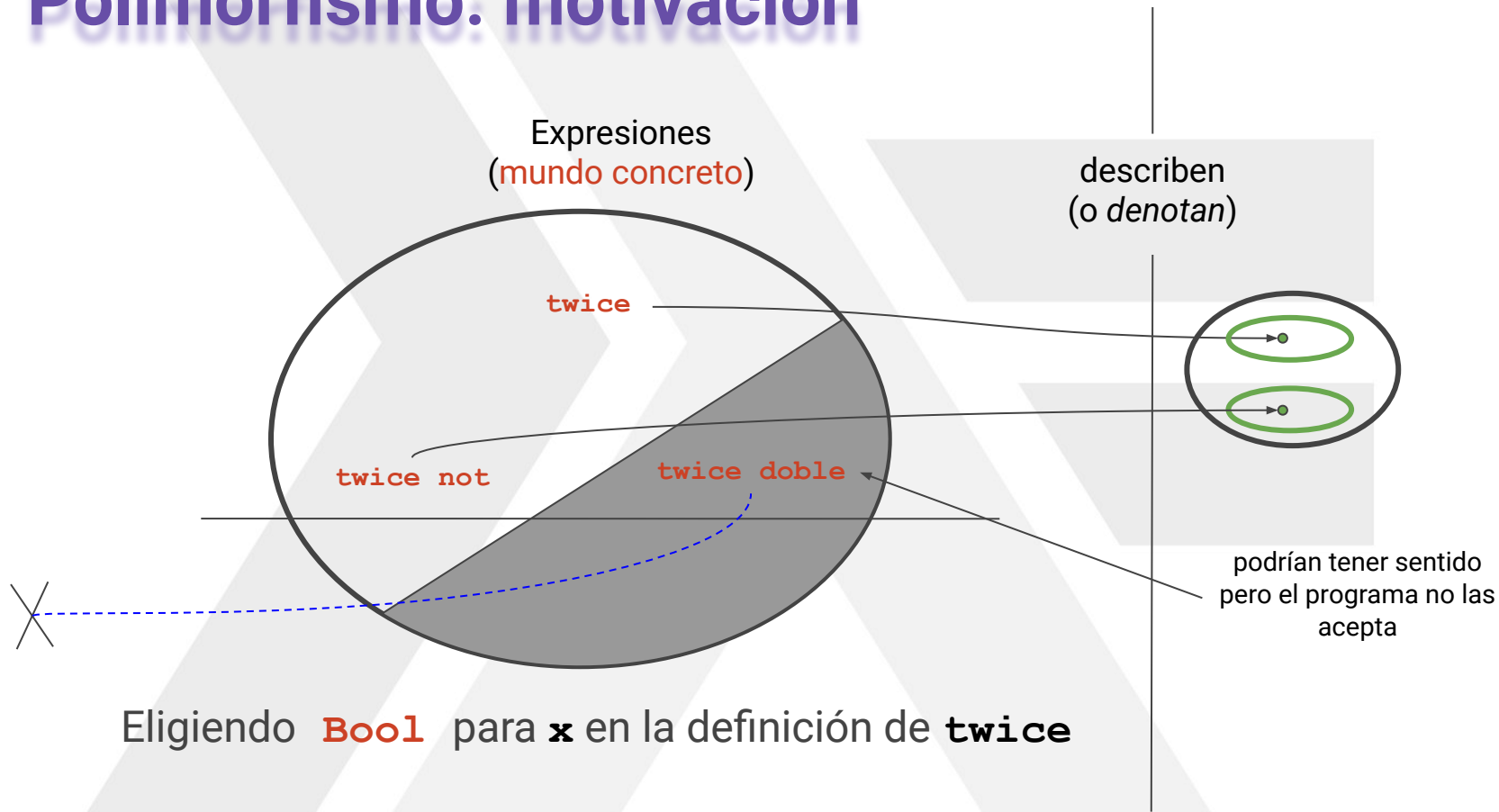
$$\begin{array}{c} \text{:: (Bool->Bool)} \quad \text{:: (Bool->Bool)} \\ \swarrow \quad \searrow \\ \text{(twice doble, twice not)} \text{ :: (} \quad \text{, Bool->Bool) } \\ \underbrace{\text{twice doble}}_{\text{:: Int->Int}} \quad \underbrace{\text{twice not}}_{\text{:: Bool->Bool}} \end{array}$$

❏ ¿Y entonces? ¿Cuál debería ser la elección para **x**?

Polimorfismo: motivación



Polimorfismo: motivación



Polimorfismo: motivación

- ❑ ¿Cuál debería ser el tipo de **twice**?

```
twice :: (Int->Int) -> (Int->Int)
```

```
twice :: (Bool->Bool) -> (Bool->Bool)
```

```
twice :: ( ?? -> ?? ) -> ( ?? -> ?? )
```

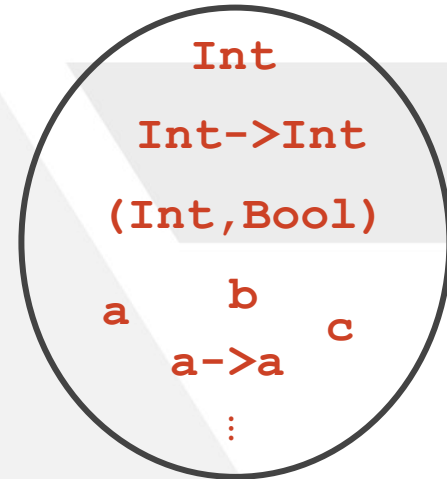
- ❑ Hay infinitas elecciones posibles para este ??
- ❑ Al forzar la elección, se invalidan infinitas otras
- ❑ ¿Cómo hacer entonces para no forzar la elección, cuando hay que elegir un tipo?

Polimorfismo

- ❑ ¿Cómo hacer para no forzar la elección del tipo?
- ❑ Necesitamos un “tipo” que permita elegir después
- ❑ ¡Un mecanismo para esto es tener ***variables de tipo***!
- ❑ Se *agregan* variables al conjunto de expresiones de tipo
 - ❑ Se utilizarán letras minúsculas, **a**, **b**, **c**, ...
 - ❑ Así, **twice :: (a -> a) -> (a -> a)**

Sistema Hindley-Milner polimórfico

- Se agregan variables a los tipos
 - Básicos
 - `Int`, `Bool`, etc.
 - Compuestos
 - `A -> B`, `(A, B)`, etc.
 - Variables (para polimorfismo)
 - `a`, `b`, `c`, ...



Expresiones
de tipo

Sistema Hindley-Milner polimórfico

- ❑ ¿Qué significa el nuevo tipo de `twice`?
 - ❑ `twice :: (a -> a) -> (a -> a)`
 - ❑ Se lee: “***twice*** es una función que dada una función que transforma elementos de ***algún tipo*** ***a*** en otros elementos de ese tipo, retorna otra función de ese mismo tipo”
 - ❑ El tipo ***a*** puede ser ***instanciado*** a diferentes tipos, en diferentes usos

```
twice f = g
  where g x = f (f x)
```

Sistema Hindley-Milner polimórfico

Instanciación de **a** según el contexto

twice :: (a -> a) -> (a -> a)

		A		B
f	twice	::	(a->a)	-> (a->a)
e	doble	::	Int->Int	A

f	twice	e	doble	::	Int->Int	B
---	--------------	---	--------------	----	----------	---

(a <- Int)

¡La variable toma diferentes valores en diferentes usos!

		A'		B'
f'	twice	::	(a'->a')	-> (a'->a')
e'	not	::	Bool->Bool	A'

f'	twice	e'	not	::	Bool->Bool	A'
----	--------------	----	------------	----	------------	----

(a' <- Bool)

Sistema Hindley-Milner polimórfico

- ❑ Definición de *polimorfismo paramétrico*
 - ❑ Un sistema de tipos posee *polimorfismo paramétrico* cuando, para expresiones que admiten infinitos tipos, puede asignar uno más general que pueda transformarse en cualquiera de ellos
 - ❑ Es una característica del sistema de tipos
 - ❑ Se dice “paramétrico” porque **a** funciona como un parámetro

Sistema H-M: ejemplo 5

Sistema Hindley-Milner polimórfico

■ Ejemplo 5: `twice twice`

■ `twice :: (a -> a) -> (a -> a)`

`twice twice ::`

■ Se plantea la pregunta

Sistema Hindley-Milner polimórfico

■ Ejemplo 5: **twice twice**

■ **twice :: (a -> a) -> (a -> a)**

::
::

->

Porque se usó la
regla de aplicación

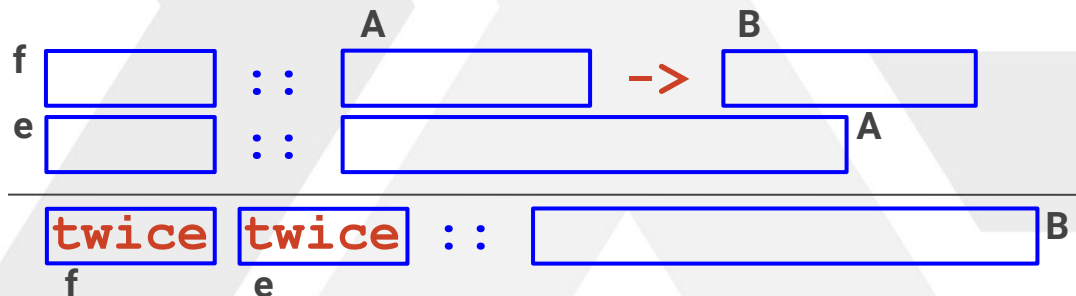
twice twice ::

■ Se determina la regla a aplicar

Sistema Hindley-Milner polimórfico

■ Ejemplo 5: **twice twice**

■ **twice** :: (a -> a) -> (a -> a)

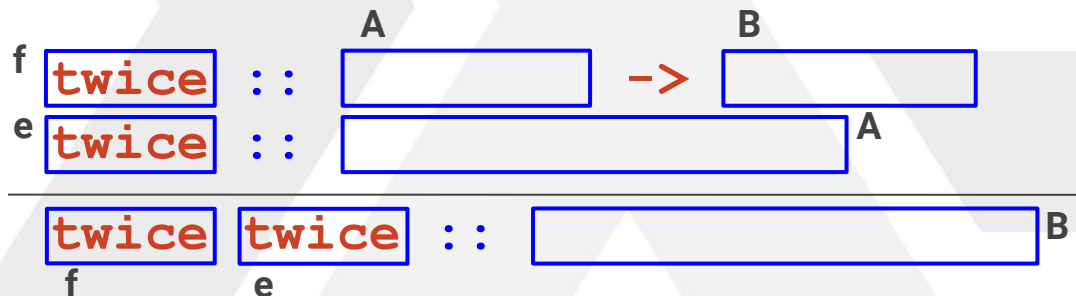


■ Se analizan cuáles son los componentes

Sistema Hindley-Milner polimórfico

■ Ejemplo 5: **twice twice**

■ **twice** :: (a -> a) -> (a -> a)



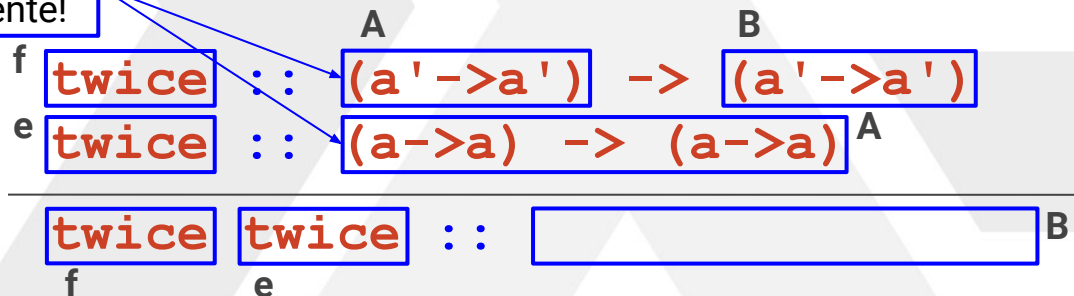
■ Se comienzan a colocar las piezas en su lugar

Sistema Hindley-Milner polimórfico

■ Ejemplo 5: `twice twice`

■ `twice :: (a -> a) -> (a -> a)`

¡En cada uso la variable es diferente!



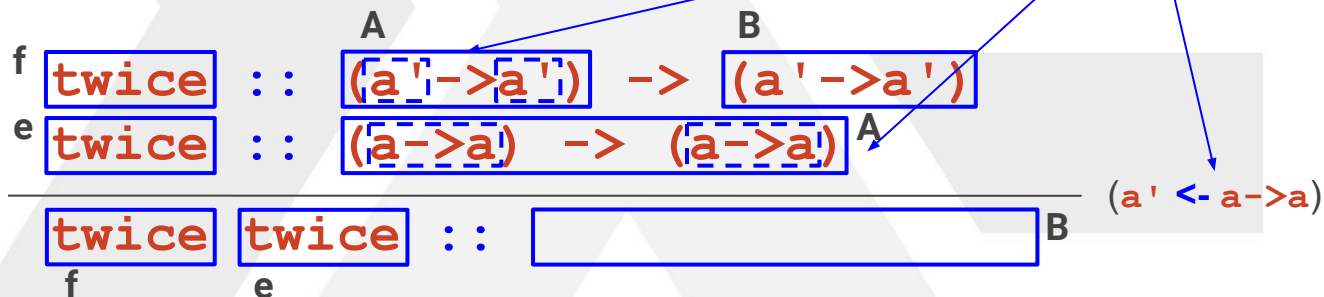
■ Se resuelven los antecedentes

Sistema Hindley-Milner polimórfico

■ Ejemplo 5: **twice twice**

■ **twice** :: (a -> a) -> (a -> a)

Instanciando **a'** se consigue que sean iguales:
¡OK!



■ Se busca verificar las condiciones
(quizás instanciando variables polimórficas)

Sistema Hindley-Milner polimórfico

■ Ejemplo 5: **twice twice**

■ **twice** :: (a -> a) -> (a -> a)

$$\frac{\begin{array}{c} \text{f} \quad \boxed{\text{twice}} :: \boxed{(a' \rightarrow a')} \rightarrow \boxed{(a' \rightarrow a')} \\ \text{e} \quad \boxed{\text{twice}} :: \boxed{(a \rightarrow a) \rightarrow (a \rightarrow a)}^A \end{array}}{\begin{array}{c} \boxed{\text{twice}} \quad \boxed{\text{twice}} :: \boxed{(a \rightarrow a) \rightarrow (a \rightarrow a)}^B \end{array}} \quad (a' \leftarrow a \rightarrow a)$$

■ Se concluye con el resultado:

twice twice :: (a->a) -> (a->a)

Sistema Hindley-Milner polimórfico

- En **twice twice** ambos usos tienen distinto tipo
 - La ocurrencia de la derecha
$$\text{twice} :: (a \rightarrow a) \rightarrow (a \rightarrow a)$$
 - El ocurrencia de la izquierda
$$\begin{aligned} \text{twice} &:: ((a \rightarrow a) \rightarrow (a \rightarrow a)) \\ &\rightarrow ((a \rightarrow a) \rightarrow (a \rightarrow a)) \\ (\text{twice} &:: (a' \rightarrow a') \rightarrow (a' \rightarrow a') \text{ con } a' \leftarrow a \rightarrow a) \end{aligned}$$
- Por eso la izquierda puede aplicarse a la derecha (coincide el tipo del parámetro con el del argumento)

Observaciones finales

- ❑ Ventajas de un sistema de tipos
 - ❑ Detección temprana de errores
 - ❑ Documentación rudimentaria
 - ❑ Oportunidades de optimización en ejecución
- ❑ Existen numerosos sistemas de tipos con diferentes características

Observaciones finales

- ❑ Sistema de tipado **fuerte** (*strong typing*)
 - ❑ Cuando una expresión tiene tipo,
NO falla *por problemas de tipos*
 - ❑ Ejemplo: Haskell
 - ❑ Contraejemplos: C, Java



Resumen

Resumen

- ❑ Validez de expresiones
 - ❑ Reglas sintácticas
 - ❑ Reglas de tipo
- ❑ Sistemas de tipos
 - ❑ Gran variedad de sistemas y características
 - ❑ Sistema H-M (con polimorfismo)
 - ❑ Numerosas ventajas
 - ❑ P.ej. el tipo ayuda a tener idea del posible comportamiento